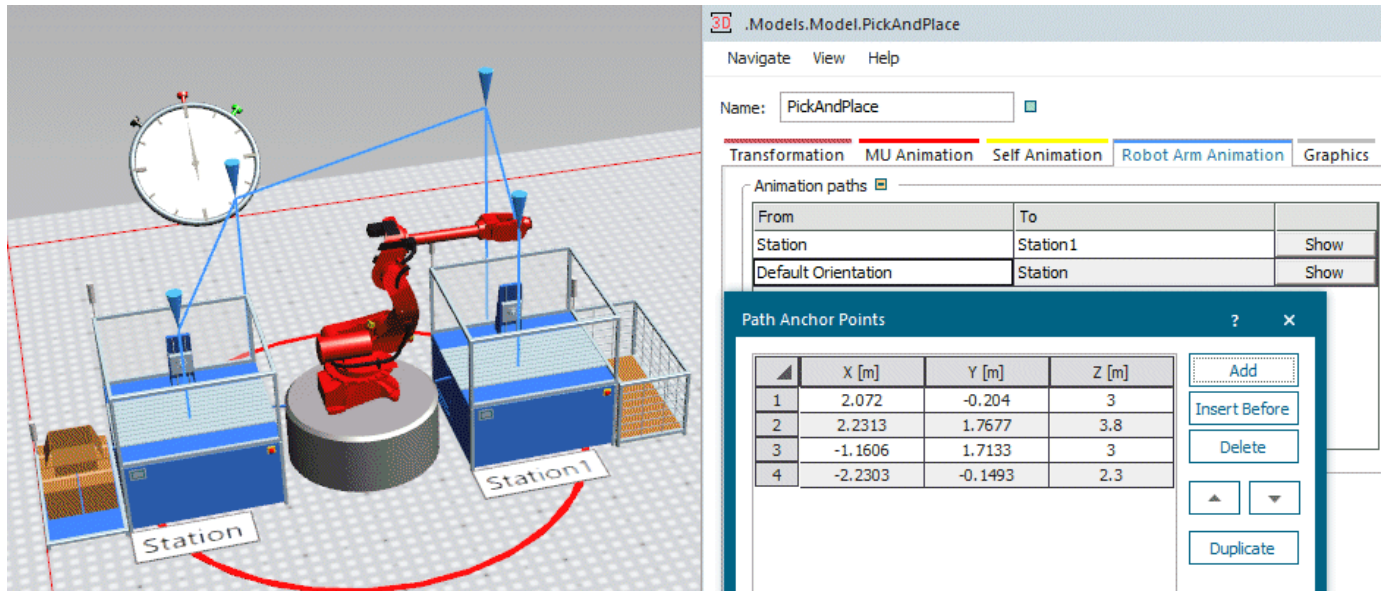




When you now run the simulation, you see that the robot arm moves above the protective cage and the indicator light without touching them.

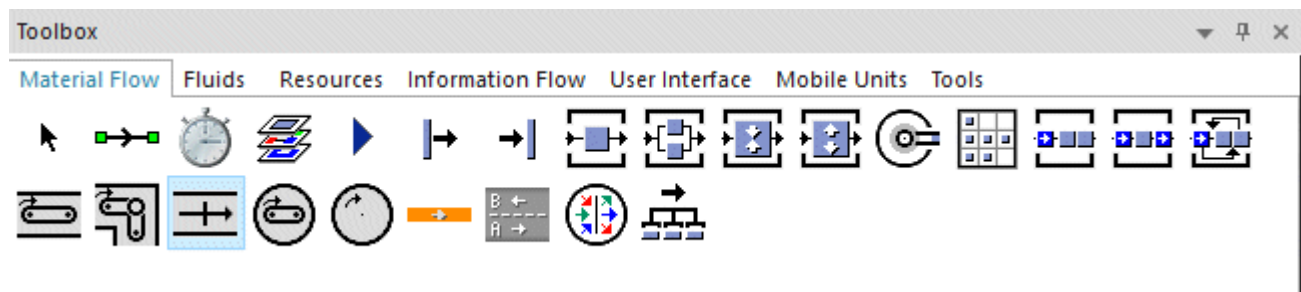
Back to Animate the Robot Arm

Convey Parts Laterally with the Converter

Convey Parts Laterally with the Converter

You can use the *Converter* to convey a part either straight through or for conveying it to the left-hand side or right-hand side.

You can insert the **Converter**  into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.



In our examples we will demonstrate how to:

- **Convey Parts Straight Through**

- Convey Parts Laterally According to Their Name
- Convey Parts According to a Strategy Control
- Send Parts to the Default Exit
- Feed Parts from a Branch Line into the Main Line

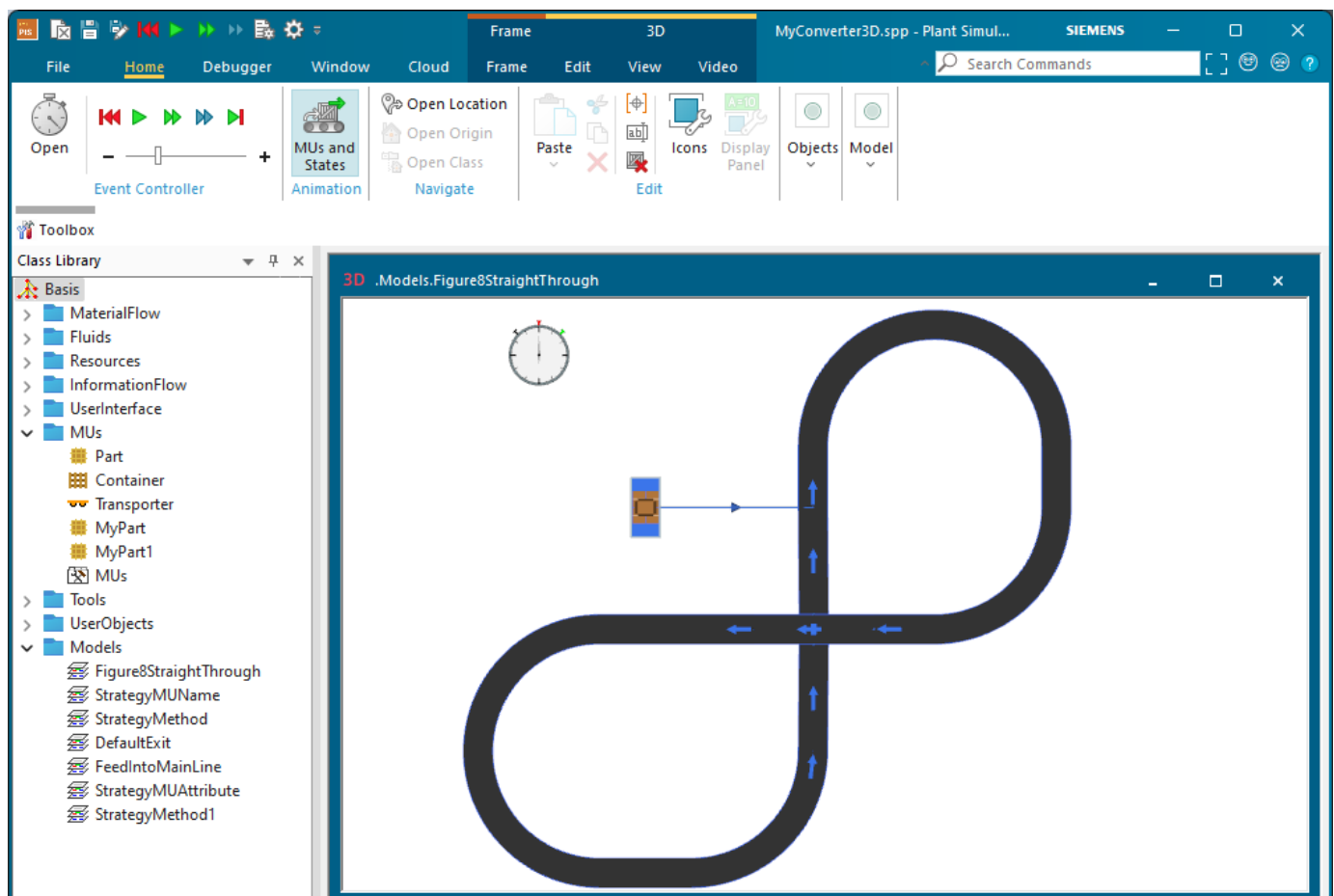
Back to [Modeling the Material Flow](#)

Convey Parts Straight Through

In this basic sample model we demonstrate how you can use the *Converter* with its default settings, i.e., without changing anything.

The *Source* produces parts and moves them on to the connected *Conveyor*. The *Converter* passes the parts straight through to the succeeding *Conveyor*.

The finished simulation model looks like this:



To create the simulation model, insert the objects so that they match the screenshot above. You can use the *Conveyors* and the *Converter* as you inserted them, i.e., without changing their settings.

.Models.Figure8StraightThrough.Converter

Navigate View Tools Tabs Help

Name: ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Statistics Energy Costs User-defin

Length: m ☐ Relative Converting Point For Length: [0..1] ☐

Width: m ☐ Width: [0..1] ☐

Speed: m/s ☐

Capacity: ☐ Automatic Stop ☐

☒ Go to Default Position ☐

Strategy: ☐

To limit the number of parts in the plant, the *Source* produces 5 parts only.

.Models.Figure8StraightThrough.Source

Navigate View Tools Help

Name: ☐ Failed ☐ Exit Locked ☐

Label: ☐ Planned ☐

Attributes Failures Controls Exit Statistics Importer User-defined

Operating Mode: ☒ Blocking ☐

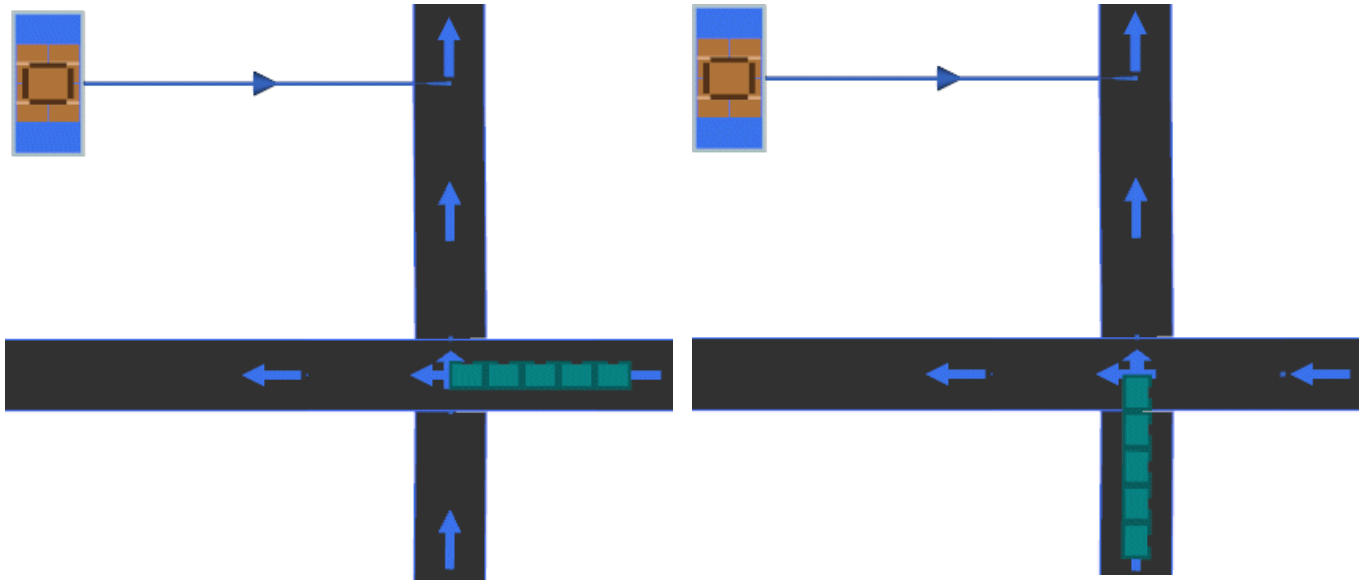
Time of Creation: ☐ Amount: ☐

Creation times: ☐

MU Selection: ☐

MU: ☐

Start the simulation and watch how the parts move on the *Conveyors* and how the *Converter* conveys them straight through in the direction of motion, first from right to left and then from bottom to top. The notch of the part is located on its right side



Back to [Convey Parts Laterally with the Converter](#)

Convey Parts Laterally According to Their Name

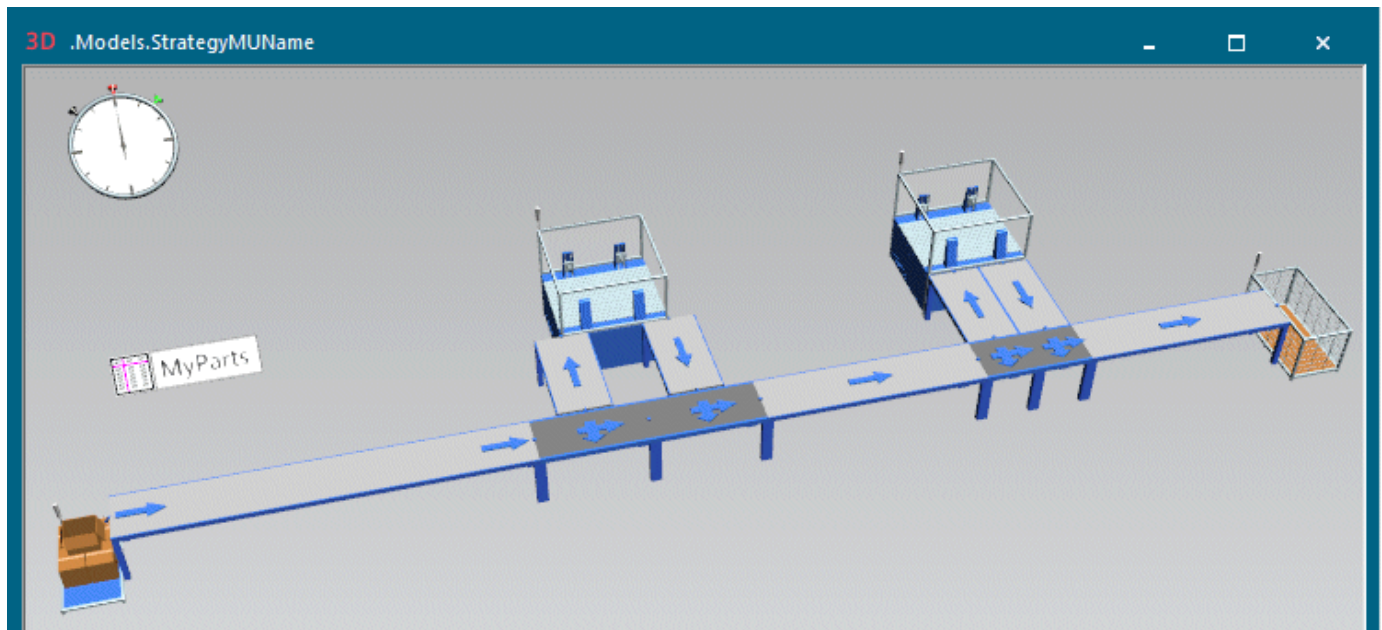
Convey Parts Laterally According to Their Name

In this sample model the *Source* produces six different types of parts with a different frequency each. The parts move over a number of *Conveyors* to two pairs of *Converters*, which—according to their name—convey them laterally onto and remove them laterally from two different processing stations.

In our examples we will:

- **[Configure the Source that Produces the Parts \[Converter\]](#)**
- **[Configure the Converters that Convey Parts to and from ProcessingA](#)**
- **[Configure the Stations ProcessingA and ProcessingB](#)**
- **[Configure the Converters that Transport Parts to and from ProcessingB](#)**

The finished simulation model looks like this:



Back to [Convey Parts Laterally with the Converter](#)

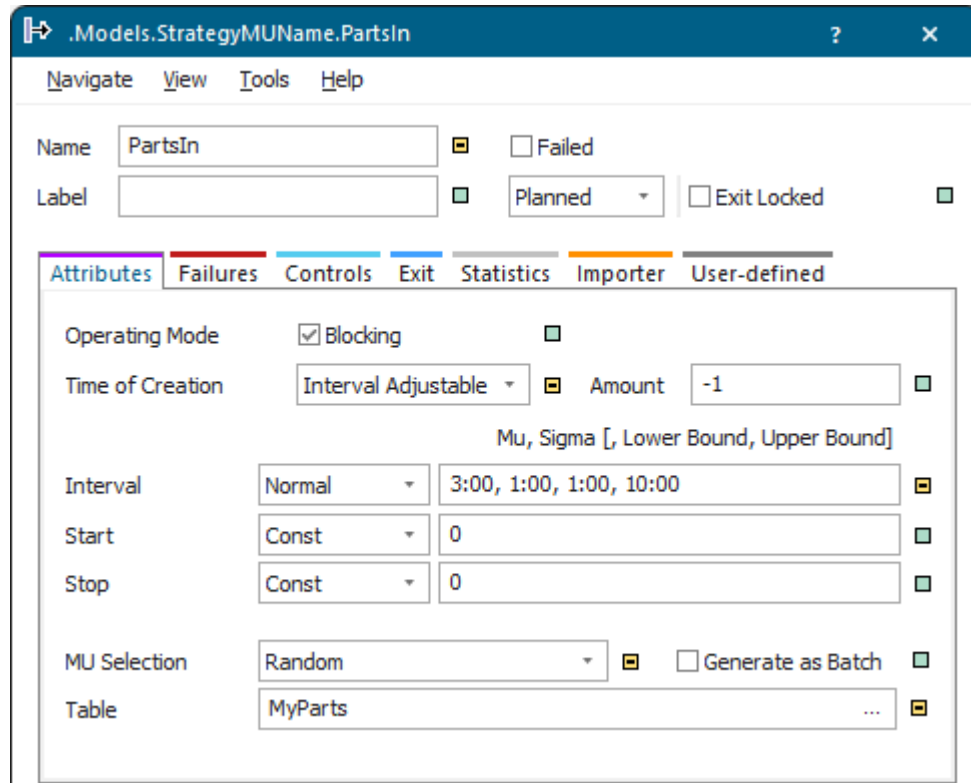
Configure the Source that Produces the Parts [Converter, laterally]

Insert and configure the *Source* which produces the parts.

Proceed as follows:

- Select the time and the interval at which the parts are to be produced.

We selected an **adjustable interval** and a **normally distributed time** in our example, as shown in the screenshot below.



- Select the type of part to be used and how these parts are to be produced.
 - We selected a **random frequency**.
 - We selected the name of the **Frequency Table**, which we inserted before, namely the *DataTable* named *MyParts*. Selecting the table automatically applied the correct format.
 - We entered the required information, as shown in the screenshot below.

	object 1	object 2	object 3	object 4	object 5
	string MU	real Frequency	integer Number	string Name	table Attributes
1	.MUs.MyPart1	3.00		A1	
2	.MUs.MyPart1	2.00		A2	
3	.MUs.MyPart1	4.00		A3	
4	.MUs.MyPart1	3.00		B1	
5	.MUs.MyPart1	2.00		B2	
6	.MUs.MyPart1	4.00		B3	
7					

The *Source* then produces the parts with the settings above and the *Converters* convey them sideways to the stations *ProcessingA* or *ProcessingB* respectively.

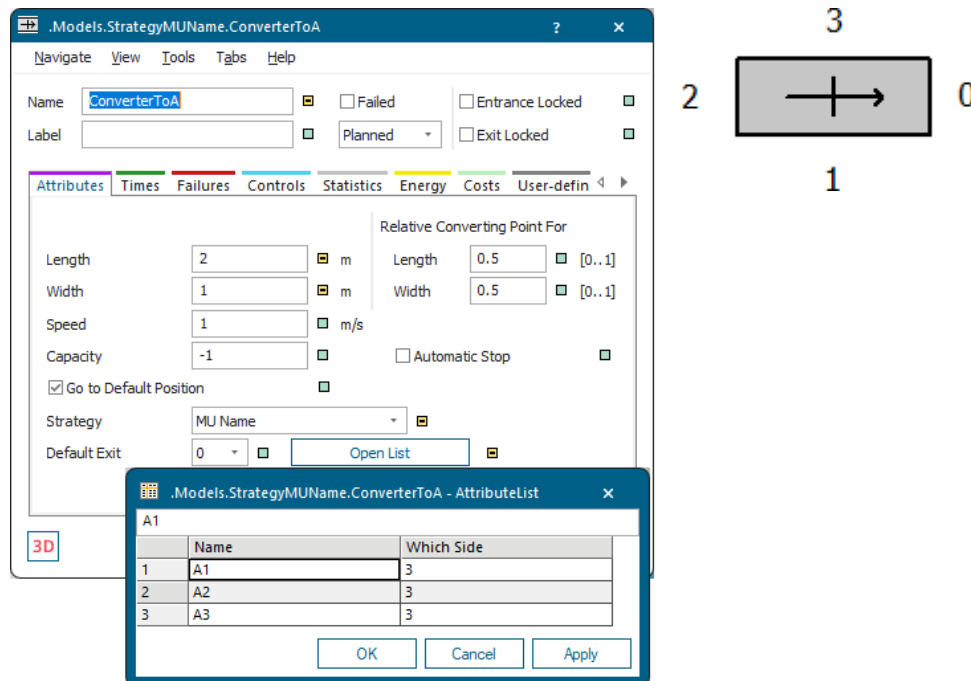
Back to Convey Parts Laterally According to Their Name

Configure the Converters that Convey Parts to and from ProcessingA

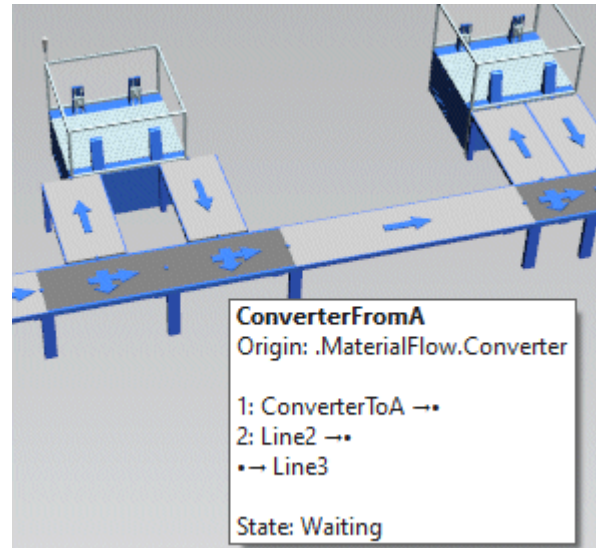
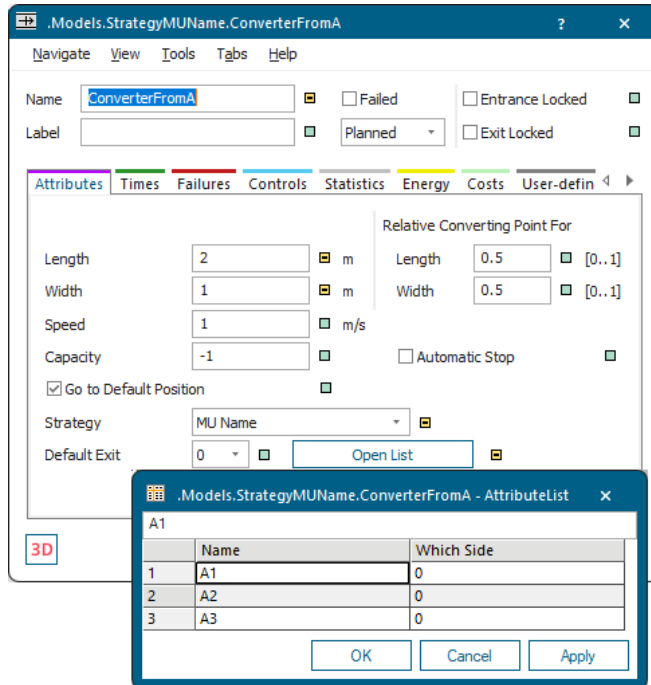
Insert and configure the *Converters* that convey the parts to and from the station *ProcessingA*.

Proceed as follows:

- As we want to convey the parts laterally to the station *ProcessingA* or *ProcessingB* respectively depending on their name, we opened *ConverterToA* and selected the **Strategy > MU Name**.
- We then clicked **Open List** and entered the names of the parts which we want to move to the station *ProcessingA* into the list. We want the parts named A1, A2, and A3 to be conveyed **laterally to the left**, so we entered their names and selected side 3 in the attribute list. As you remember, the numbers designate the side of the *Converter* at which the part exits.



- As we want to convey the parts arriving from *ProcessingA* straight on, we opened *ConverterFromA* and selected the **Strategy > MU Name**.
- We then clicked **Open List** and entered the names of the parts which we want to move on into the list. We want the parts named A1, A2, and A3 to be conveyed **straight on**, so we entered their names and selected side 0 in the attribute list.



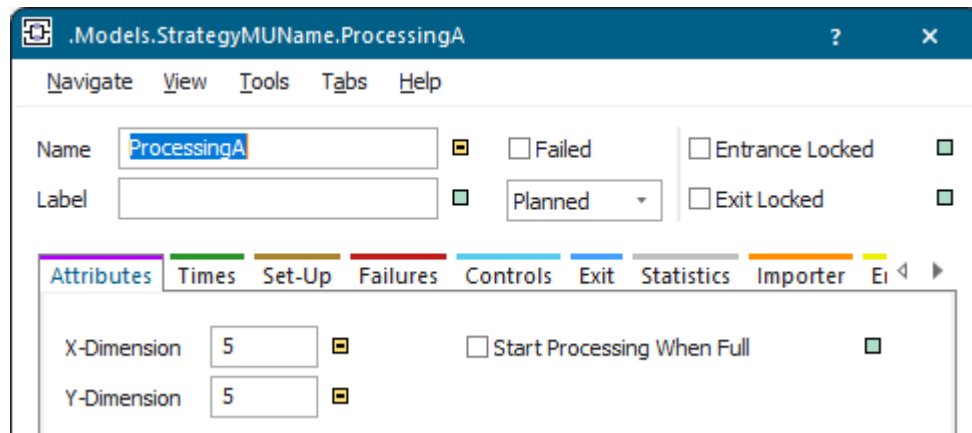
Back to Convey Parts Laterally According to Their Name

Configure the Stations ProcessingA and ProcessingB

Insert and configure the processing stations, which process the parts for the specified time before feeding them back into the factory.

Proceed as follows:

- Insert two stations of type *ParallelStation* in our sample model.
- Enter the capacity of the processing stations. We entered 5 as the **X-Dimension** and as the **Y-Dimension** respectively. This results in 25 processing places on each station.



- Enter a **Processing Time**. We decided to use a **normally distributed time** with the settings below.

.Models.StrategyMUName.ProcessingA

Navigate View Tools Tabs Help

Name: ProcessingA ☐ Failed ☐ Entrance Locked ☒

Label: ☒ Planned ☐ Exit Locked ☒

Attributes Times Set-Up Failures Controls Exit Statistics Importer Ei

Mu, Sigma [, Lower Bound, Upper Bound]

Processing Time: Normal 10:00, 5:00, 0:30, 1:00:00 ☒

☒ Automatic Processing ☐

Set-up Time: Const 0 ☒

Recovery Time: Const 0 ☒

Recovery Time Starts: When part enters ☒

Cycle Time: Const 0 ☒

Back to Convey Parts Laterally According to Their Name

Configure the Converters that Transport Parts to and from ProcessingB

Insert and configure the *Converters* that convey the parts to and from the station *ProcessingB*.

Proceed as follows:

- As we want to convey the parts laterally to the station *ProcessingB* depending on their name, open *ConverterToB* and select the **Strategy > MU Name**.
- Then, click **Open List** and enter the names of the parts which we want to move to the station *ProcessingB* into the list. We want the parts named B1, B2, and B3 to be conveyed **laterally to the left**, enter their names and select side 3 in the attribute list. As you remember, the numbers designate the side of the *Converter* at which the part exits.

.Models.StrategyMUName.ConverterToB

Navigate View Tools Tabs Help

Name: ConverterToB ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned ☐ Exit Locked ☐

Attributes Times Failures Controls Statistics Energy Costs User-defin

Length: 1 m ☒ Length: 0.5 [0..1]

Width: 1 m ☒ Width: 0.5 [0..1]

Speed: 1 m/s ☒

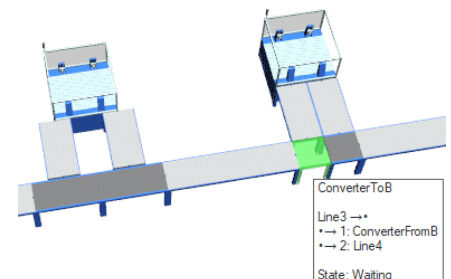
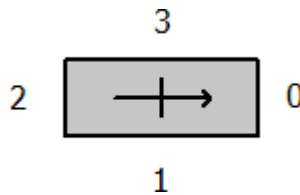
Capacity: -1 ☒ Automatic Stop ☐

☒ Go to Default Position

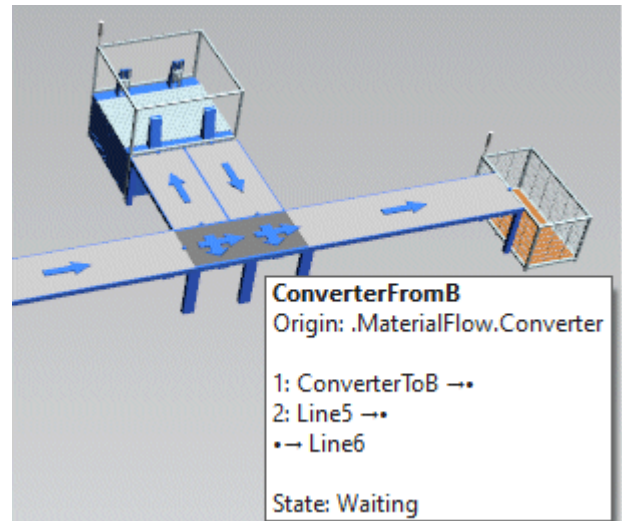
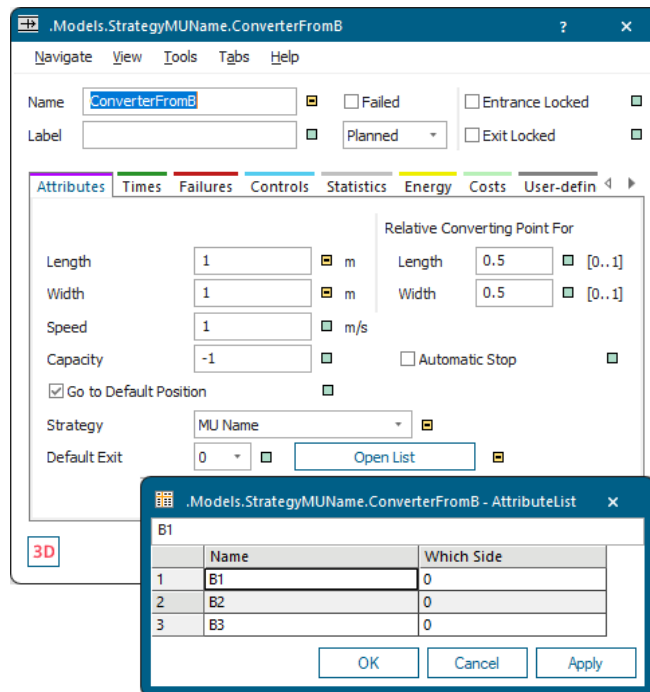
Strategy: MU Name ☒

Default Exit: 0 ☐ Open List ☒

Name	Which Side
B1	3
B2	3
B3	3



- As we want to convey the parts arriving from *ProcessingB* straight on, open *ConverterFromB*, and select the **Strategy > MU Name**.
- Then, click **Open List** and enter the names of the parts which you want to move on into the list. We want the parts named B1, B2, and B3 to be conveyed **straight on**, so enter their names and select the side 0 in the attribute list.



- Start the simulation and watch how the *Converters* convert the parts laterally to the left and straight on depending on their name. To better recognize that they do this correctly, we drew an icon with an arrow pointing in the direction of motion.

You will notice that the right hand side of the part points downwards after it has moved from *ConverterToB* into the processing station. This is because moving the part laterally to the left rotates it 90 degrees clockwise.

Once the part moves from the processing station to *ConverterFromB*, it is rotated another 90 degrees clockwise, thus moving backward toward the *Drain*, which removes it from the installation.

Back to Convey Parts Laterally According to Their Name

Conveying Parts According to a Strategy Control

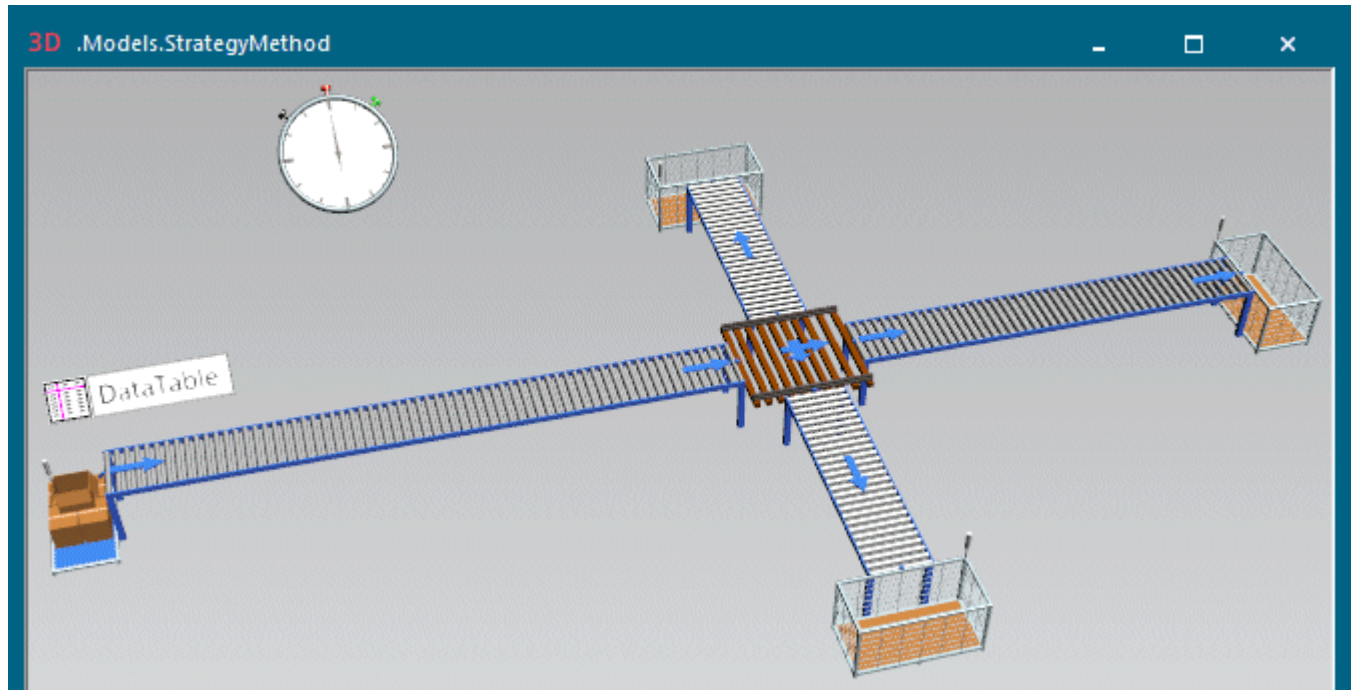
Convey Parts According to a Strategy Control

In this sample model the *Source* produces 10 parts each of three different types. It moves the parts to a *Converter*, which conveys them laterally to the left, laterally to the right, or straight through according to their names. We programmed where exactly each type goes in a **Strategy Control**.

In our examples we will:

- **Configure the Source that Produces the Parts [Converter, laterally]**
- **Configure the Converter that Conveys the Parts**

The finished simulation model looks like this:



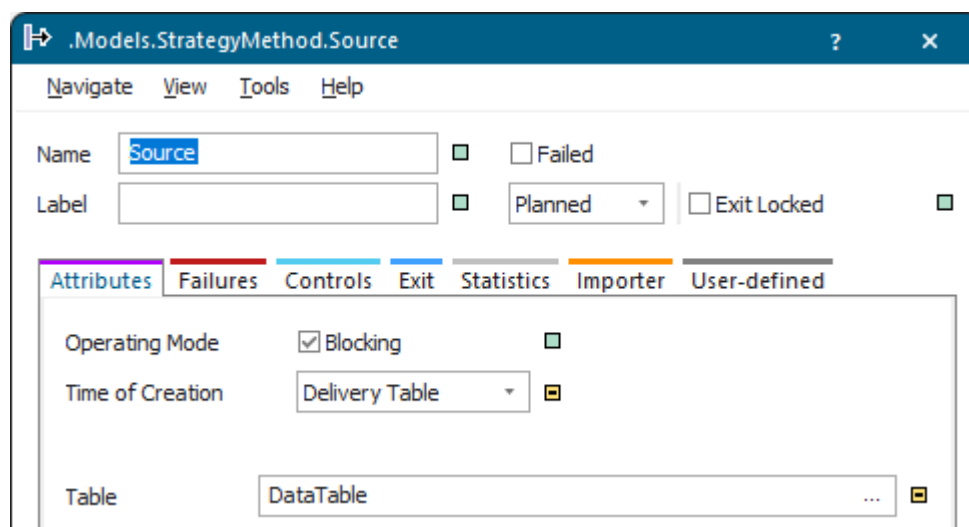
Back to **Convey Parts Laterally with the Converter**

Configure the Source that Produces the Parts [Converter]

Insert and configure the *Source* which produces the parts.

Proceed as follows:

- Select when and how many parts of a certain type are to be produced.
Select a **Delivery Table** in our example, as shown in the screenshot below.



- Then select the name of the delivery table, which we inserted before, namely the *DataTable*. Selecting it automatically applied the correct format.

Enter the required information, as shown the screenshot below.

	time	object	integer	string	table
	1	2	3	4	5
string	Delivery Time	MU	Number	Name	Attributes
1	0.0000	.MUs.Part	10	A	
2	8.0000	.MUs.Part	10	B	
3	25.0000	.MUs.Part	10	C	
4					
5					

The *Source* then produces the parts with the settings above and the *Converter* conveys them to the connected objects.

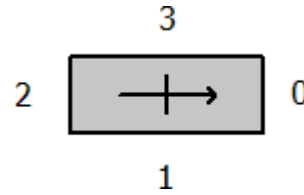
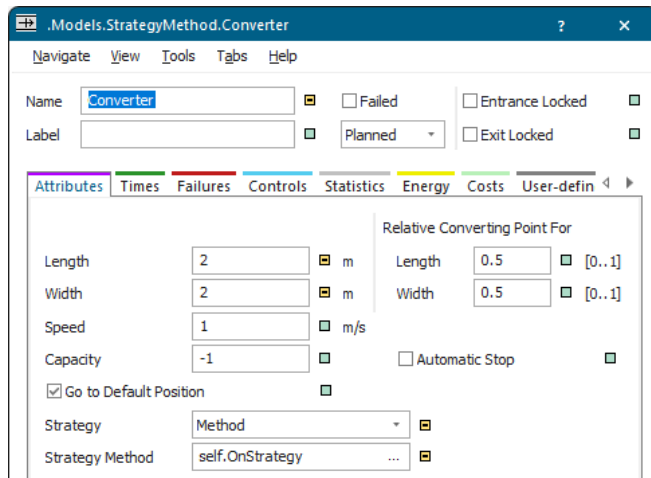
Back to [Convey Parts According to a Strategy Control](#)

Configure the Converter that Conveys the Parts

Insert and configure the *Converter* that conveys the parts.

Proceed as follows:

- As we want to convey the parts—depending on their type—laterally or straight on to the stations connected to the *Converter*, select the **Strategy > Method**.



- To create the control, right-click into the text box **Strategy Method** and select **Create Control**. *Plant Simulation* then enters **self.OnStrategy** and opens the method.

Then, type in the following source code. It conveys the parts named A laterally to the left, i.e., to exit 3 of the *Converter*, the parts named B laterally to the right, i.e., to exit 1, and the parts named C straight on, i.e., to exit 0 of the *Converter*.

```
param entranceNo: integer
if @.name = "C"
    ?.ExitForMU := 0 /* number of the exit of the converter*/
elseif @.name = "A"
    if entranceNo = 2
        ?.ExitForMU := 3
    else
        ?.ExitForMU := 0
    end
else
    if entranceNo = 2
        ?.ExitForMU := 1
    else
        ?.ExitForMU := 0
    end
end
```

- Start the simulation and watch how the *Converter* first moves the 10 parts named A laterally to the left, the 10 parts named B laterally to the right, and the 10 parts named C straight on.

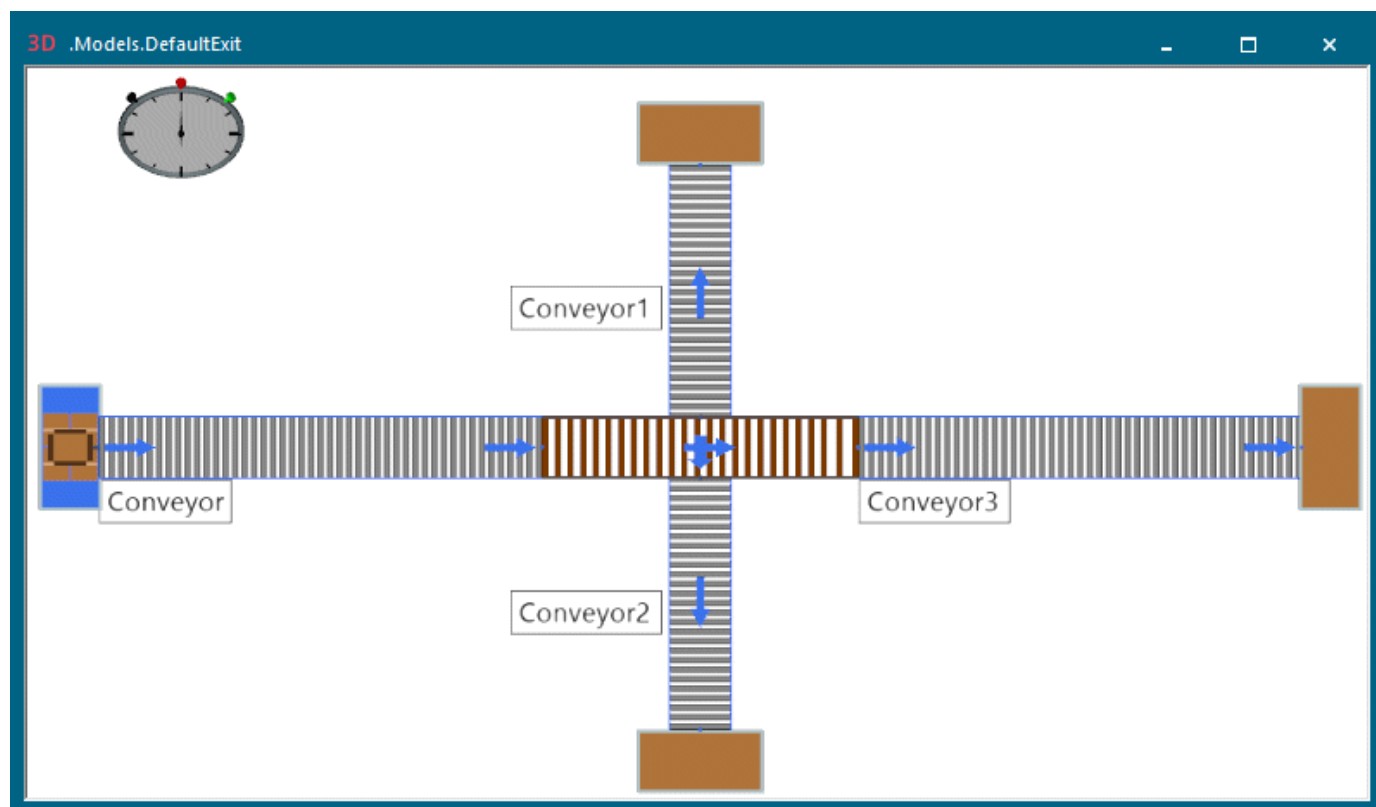
Back to **Convey Parts According to a Strategy Control**

Send Parts to the Default Exit

In this sample model the *Converter* sends all arriving parts to the **Default Exit**, which you select.

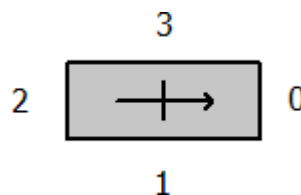
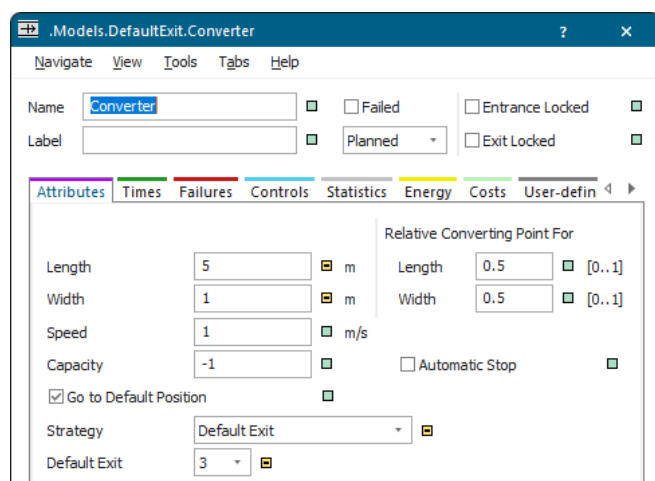
To create the simulation model, insert the objects so that they match the screenshot below.

The finished simulation model looks like this:

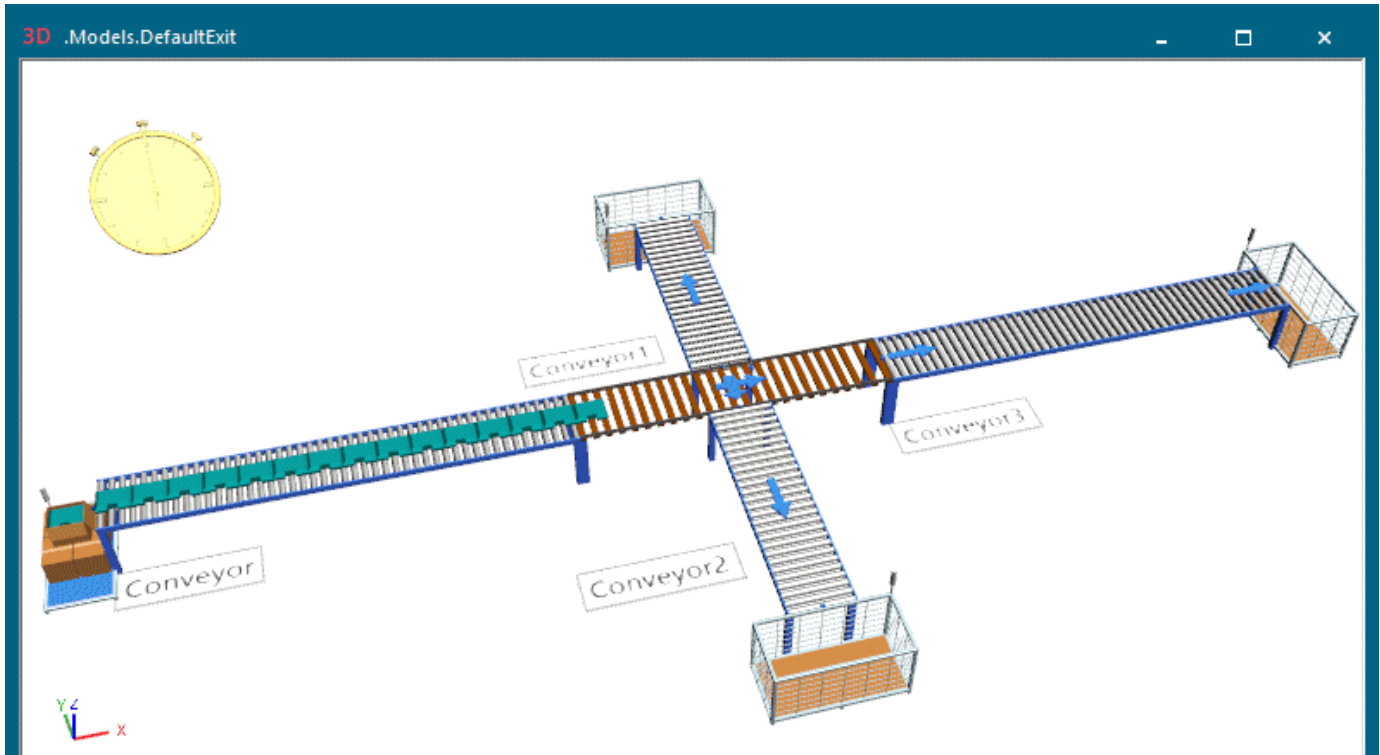


To make the *Converter* move the parts to the default exit on the respective succeeding *Conveyor*, open the dialog of the *Converter*.

- Select the **Strategy > Default Exit**.
- Select the number of the **Default Exit**. We selected **3** as we want all parts to exit upward. As you surely remember, the exits are numbered like this:



Start the simulation and watch how the parts move on the *Conveyors* and how the *Converter* conveys them to the **exit** with the number you selected, **3** in our case.



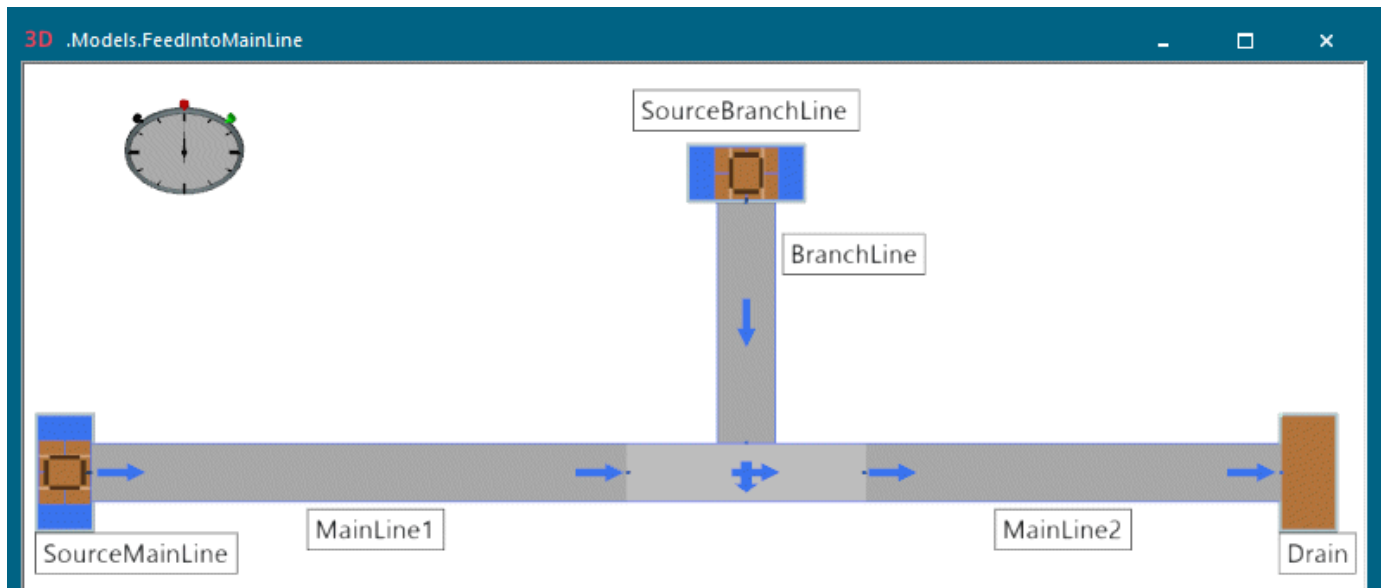
Back to [Convey Parts Laterally with the Converter](#)

Feed Parts from a Branch Line into the Main Line

In this sample model the *Converter* feeds parts from a branch line into the main line, once the gap, which you enter, is big enough to accommodate the part.

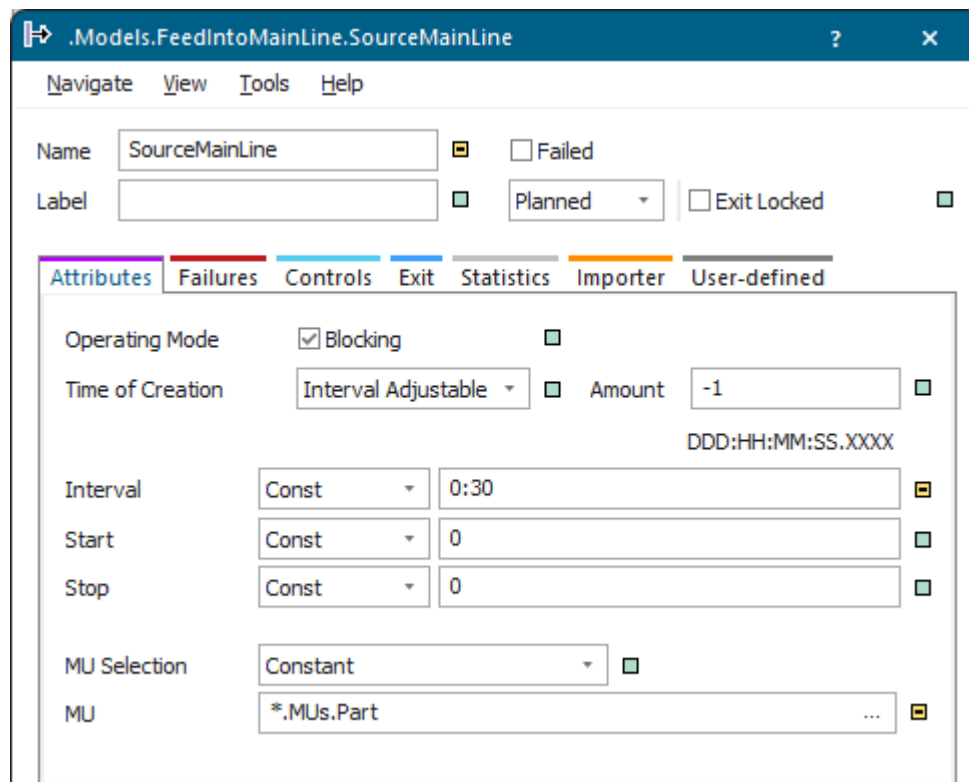
To create the simulation model, insert the objects so that they match the screenshot below.

The finished simulation model looks like this:



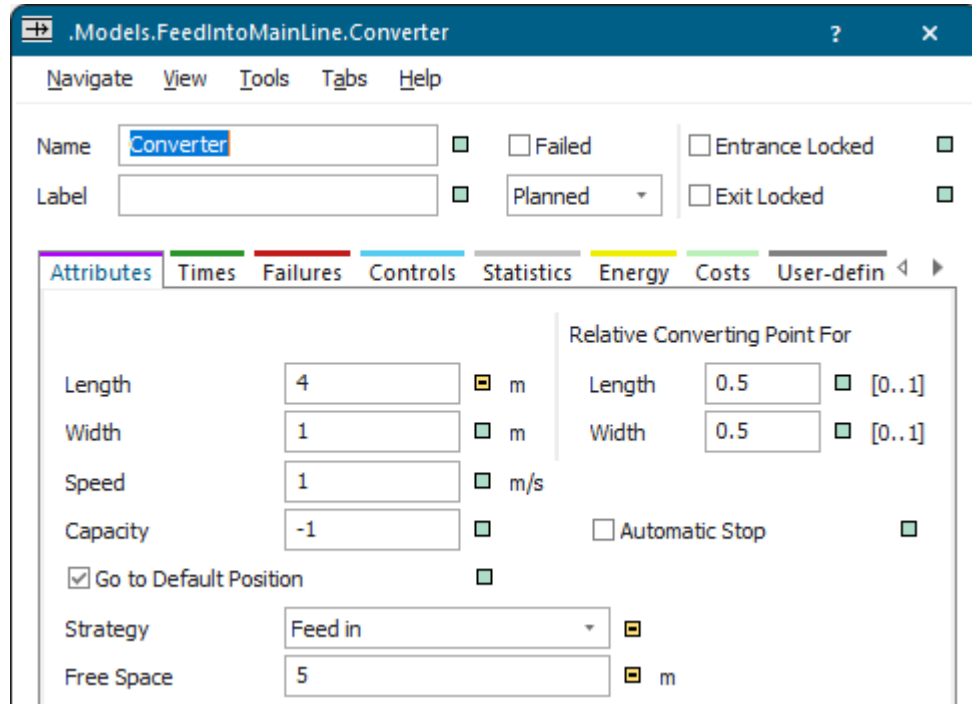
Insert and configure the *Source* which produces the parts moving on the main line:

- Select when the parts are to be produced. We entered a **constant interval** of 30 seconds, meaning that the *Source* produces a part every 30 seconds.
- Select which part type is going to be produced. We selected the part named *Part*, to which we added a direction arrow in its icon.

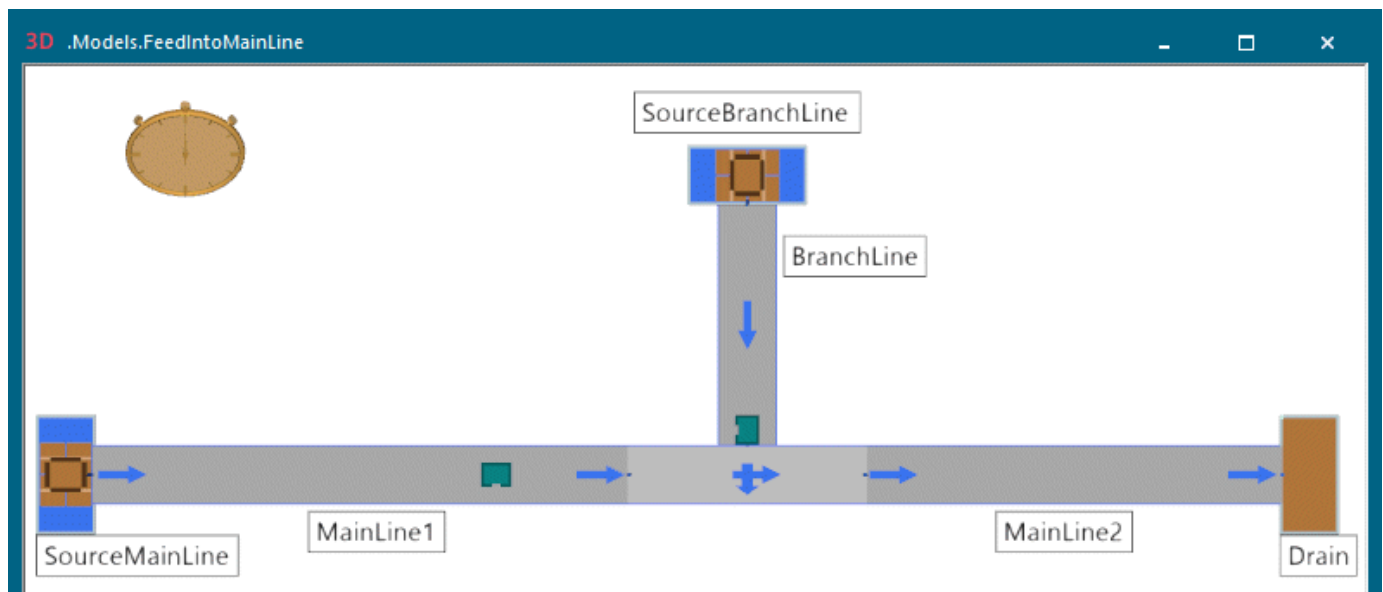


To make the *Converter* feed the parts from the branch line into the main line, provided the gap between the parts on the main line is big enough, open the dialog of the *Converter*.

- Select the **Strategy > Feed in**.
- Enter the size of the gap (**Free Space**) between the parts. We entered a gap of 5 meters.



Start the simulation and watch how the *Converter* moves the parts from the branch line into the main line and conveys them with the orientation in which they were fed in.

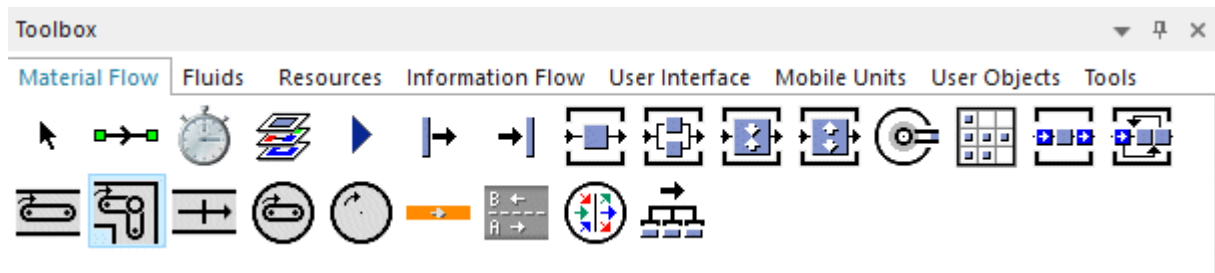


Back to Convey Parts Laterally with the Converter

Change the Conveying Direction with the AngularConverter

Use the *AngularConverter* to change the conveying direction of the mobile objects from lengthwise to crosswise conveyance or from crosswise to lengthwise conveyance. The *AngularConverter* can contain a single part at any one time.

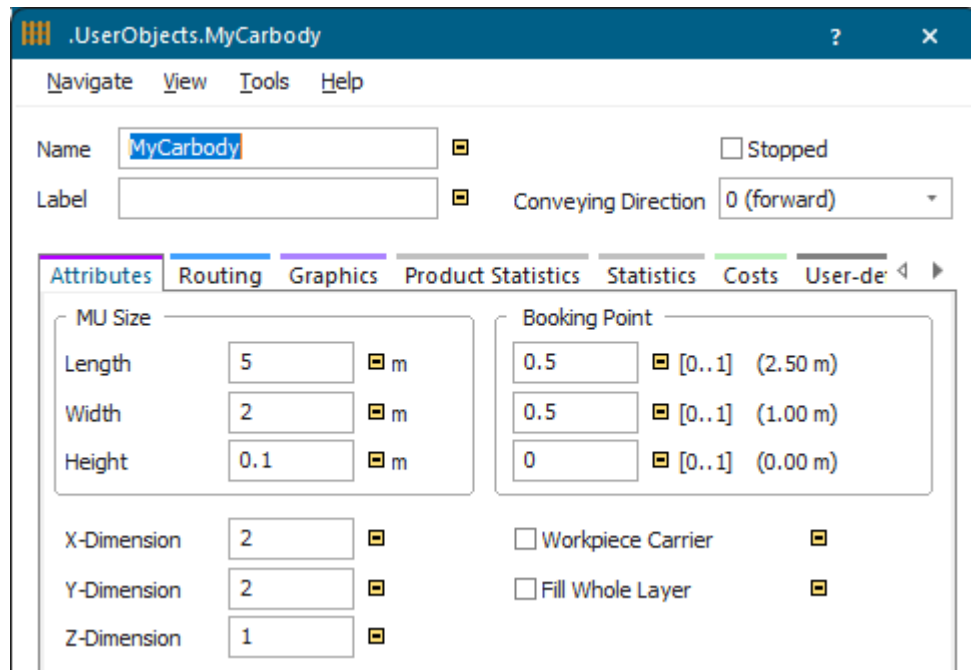
You can insert the **AngularConverter** into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.



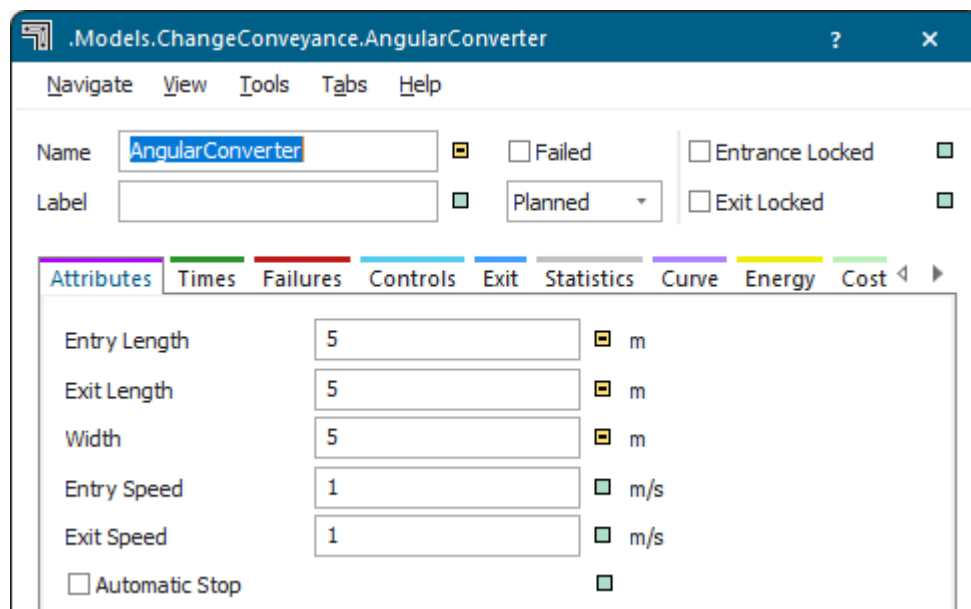
In the sample model the *Source* named *PartsIn* produces parts of type *MyCarbody* and moves them to the *AngularConverter*. It in turn changes their **conveying direction**. The length, which the part takes up on the *AngularConverter*, then changes from **MU Length** to **MU Width**. This process uses up the **Moving Time**. Once the part has exited the *AngularConverter*, it returns to its original position. It then is ready for the next *MU*.

To create the simulation model, proceed as follows:

- First, insert and configure the *Source*. Select the part which it is to produce. We named our *Source* *PartsIn* and selected the part *MyCarbody*, which we created in the folder *UserObjects*.



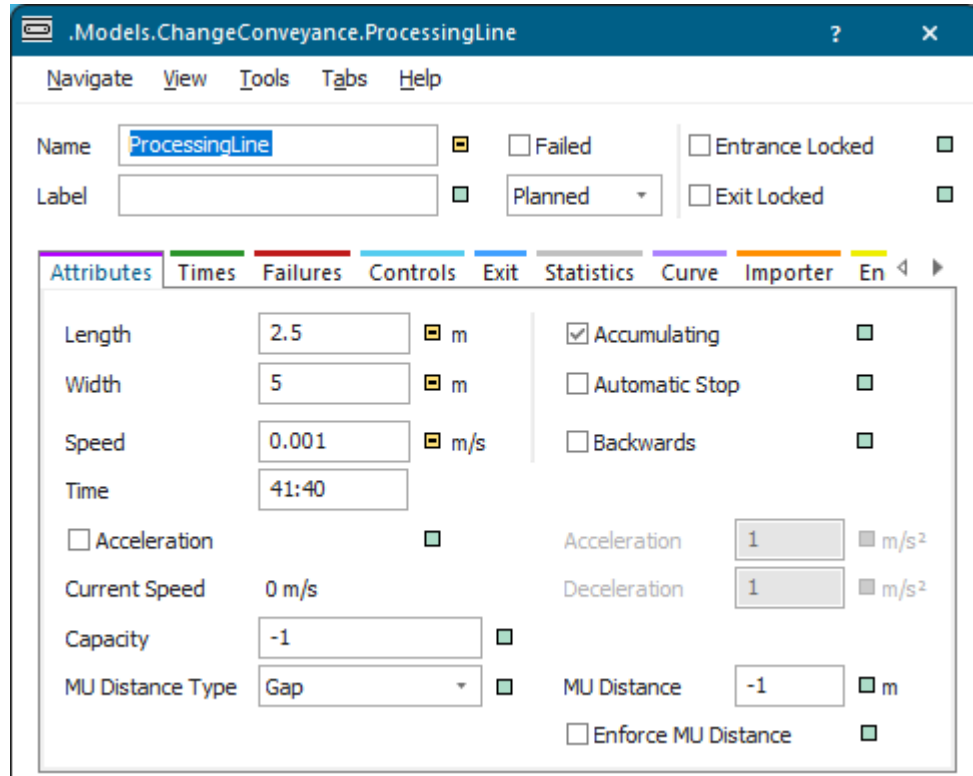
- Insert the *Conveyor* that feeds the parts to the *AngularConverter*. You can make it as long as you like, but you do not have to change any of the other settings. We made our feeder line **10** meters long. We also changed some **curve** settings for display reasons to better accommodate our part named *MyCarbody* on the *Conveyor*.
- Insert the first *AngularConverter* and configure it. Enter an **Entry Length** and an **Exit Length** of 5 meters each.



- Insert the *ProcessingConveyor* into which the *AngularConverter* moves the parts and configure it.
 - We entered a **Length** of 2.5 meters.

- We entered a **Speed** of 0.001 meters per second to make the part stay on the processing line instead of being moved on continually.

Plant Simulation also recomputes the time when you click **Apply** and/or **OK**. In our case a time of **Speed** of 0.001 meters per second over a **Length** of 2.5 meters corresponds to a **Time** of 41 minutes and 40 seconds and vice versa.



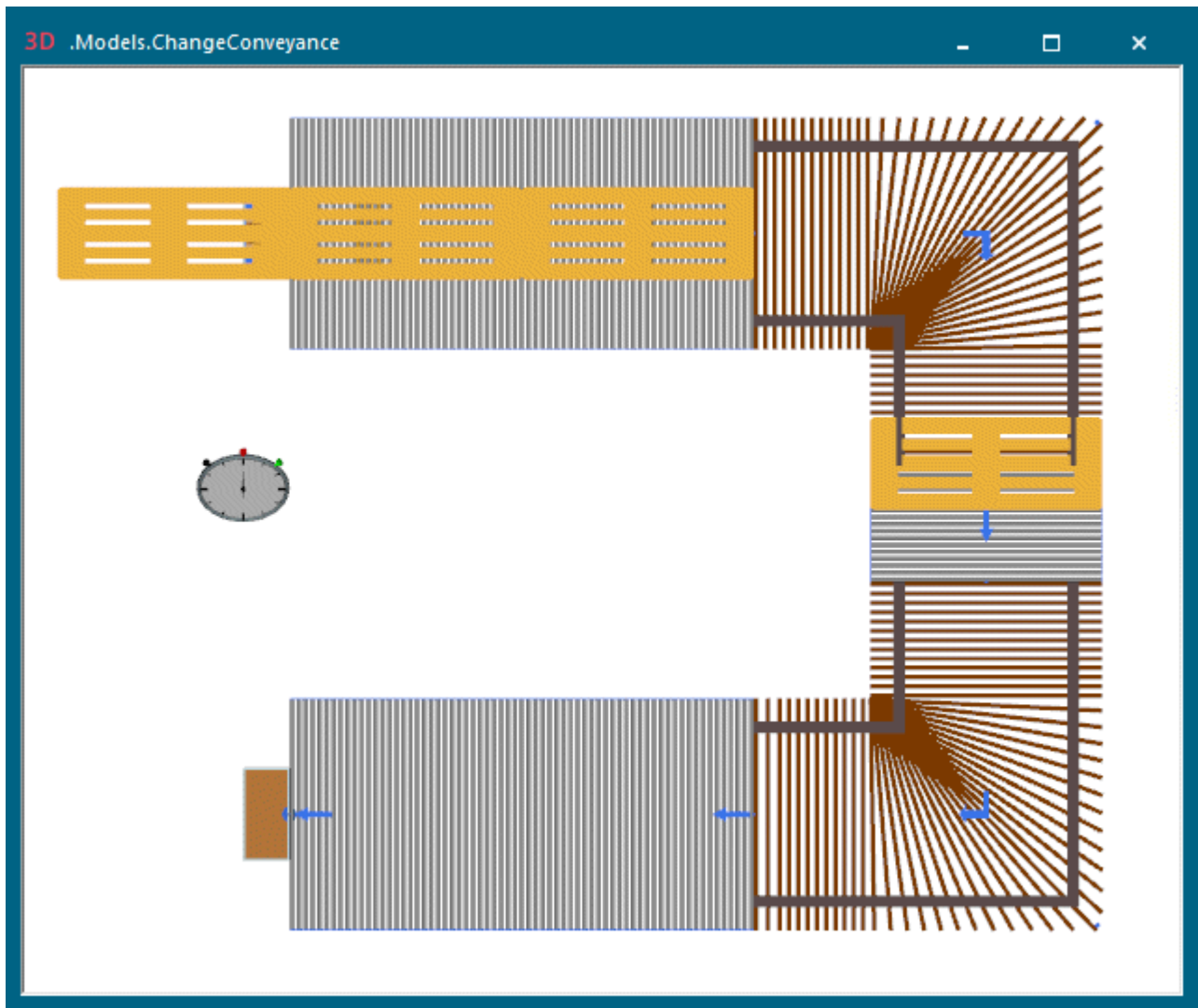
- To finish the simulation model, insert the second *AngularConverter*, another *Conveyor*, and the *Drain*, which removes the parts from the plant. For the *AngularConverter* and the *Conveyor* you can use the same settings as for the *Conveyor* and the *AngularConverter*, which you inserted above. Finally, connect all objects with *Connectors*.
- To make the model visually more pleasing, move the objects closer to each other so that the *Connectors* are not visible. Then, reduce the simulation speed by dragging the speed slider to the left, start the simulation, and watch how the car bodies move over the stations.

Double-click a part when it has reached the point at which the conveying direction changes on the *AngularConverter* at the top. Up to this point the **Conveying Direction** is **forward**.

After this point the **Conveying Direction** is **lateral right** until the part reaches the point at which the conveying direction changes again.

After this point the **Conveying Direction** changes to **backward** until the part reaches the *Drain*.

We can select other appearances than the default settings for the *Conveyors* and the *AngularConverter*. As we do not have any 3D graphic for the car-bodies, we use the pallet with the same dimensions.



Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

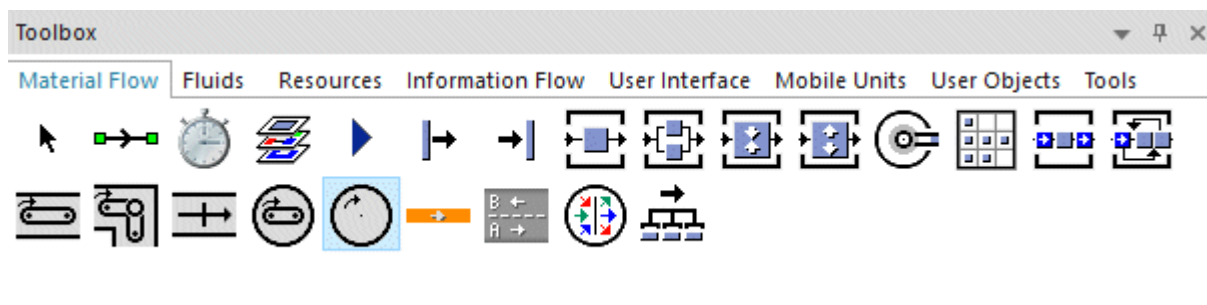
Back to [Modeling the Material Flow](#)

Aligning and Shrink-Wrapping Parts with the Turnplate

Align and Shrink-Wrap Parts with the Turnplate

You can use the *Turnplate* to model a rotating platform, which rotates the loaded part and ensures the uniform orientation of the leaving parts. A typical example is parcel shipping where all parcels have to be rotated to a uniform direction so that a scanner can automatically read the bar-code holding the address information.

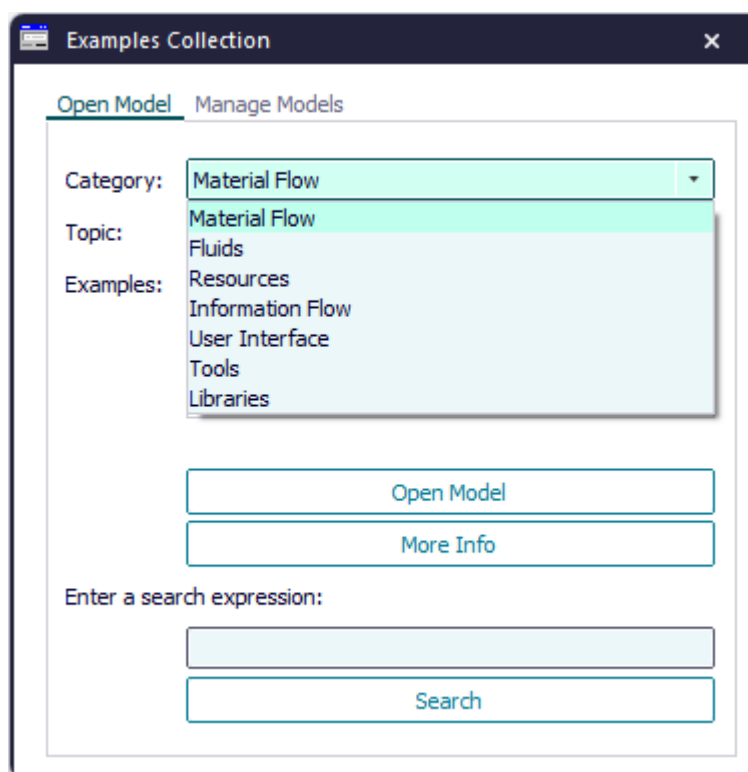
You can insert the **Turnplate** into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.



In our examples we will demonstrate how to:

- [Align Parts with the Turnplate](#)
- [Model a Shrink Wrapper](#)

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

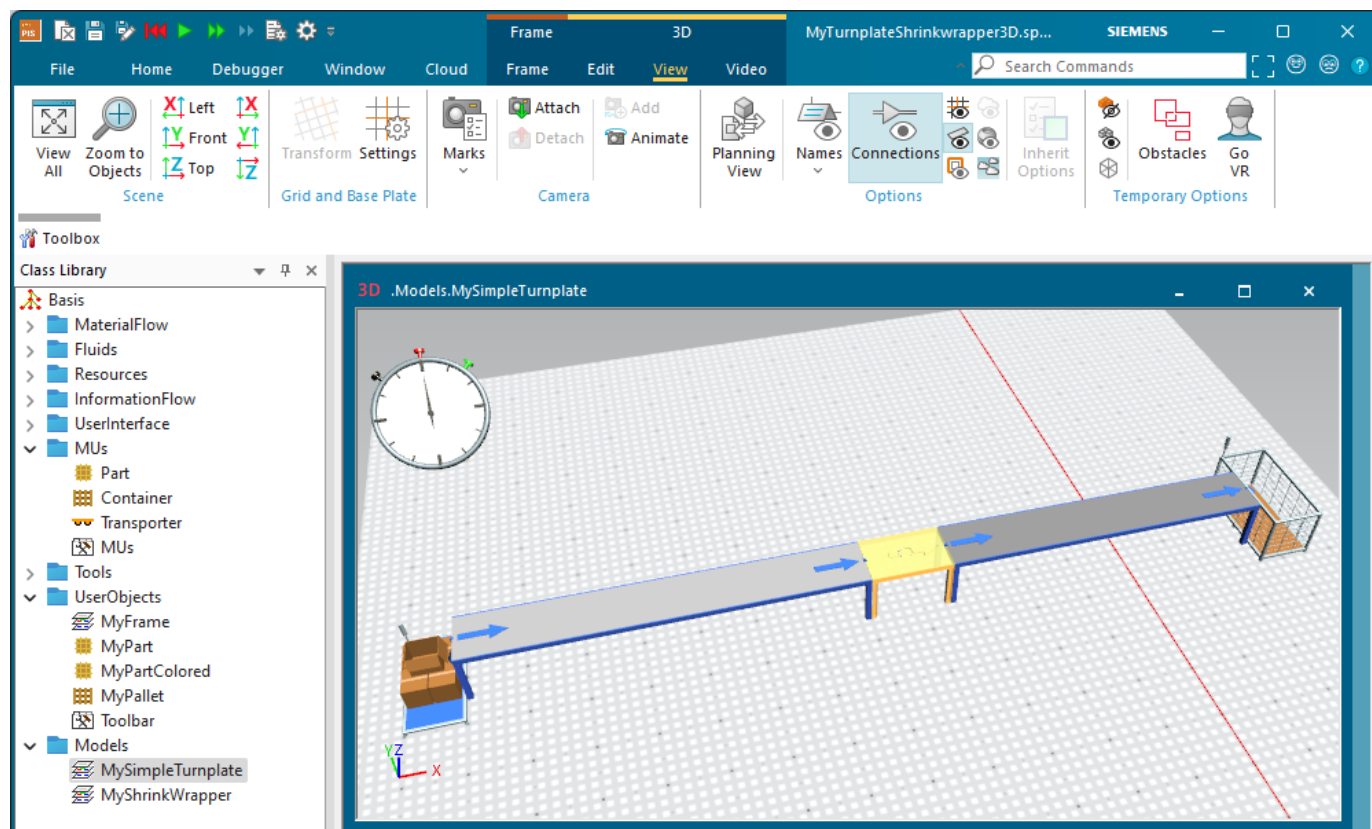


Back to [Modeling the Material Flow](#)

Align Parts with the Turnplate

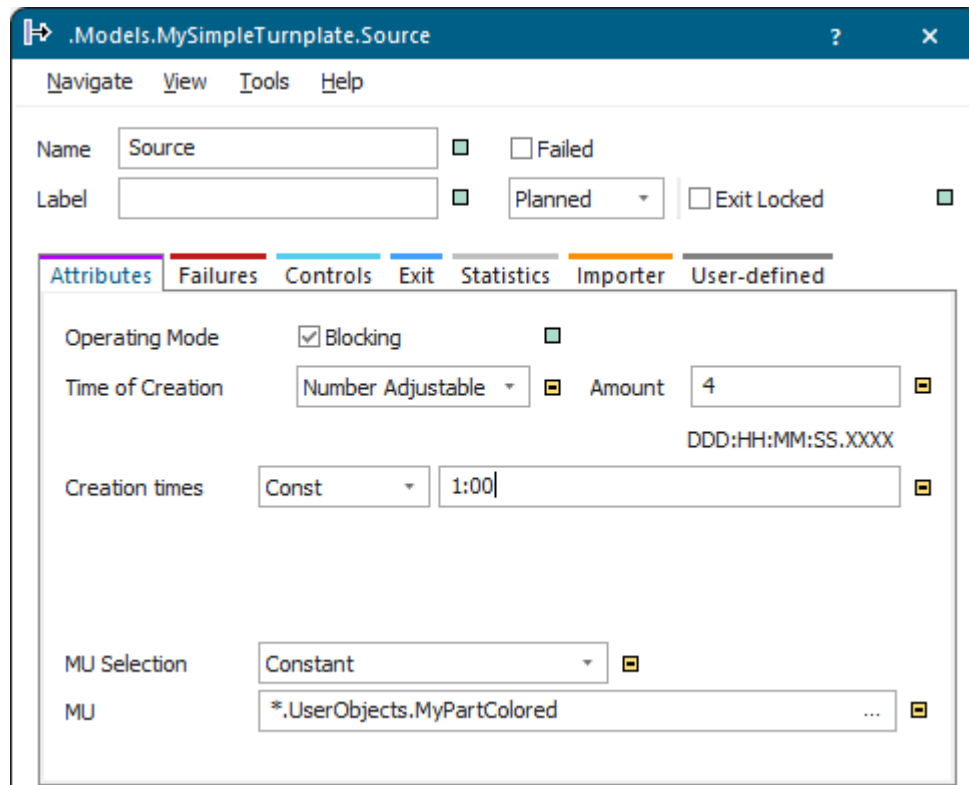
In this sample model the *Source* produces four parts of type *MyPart* and moves them onto the feeder line of the *Turnplate*. It in turn rotates each part by 90 degrees to the right and moves it on to its successor.

The finished simulation model looks like this:

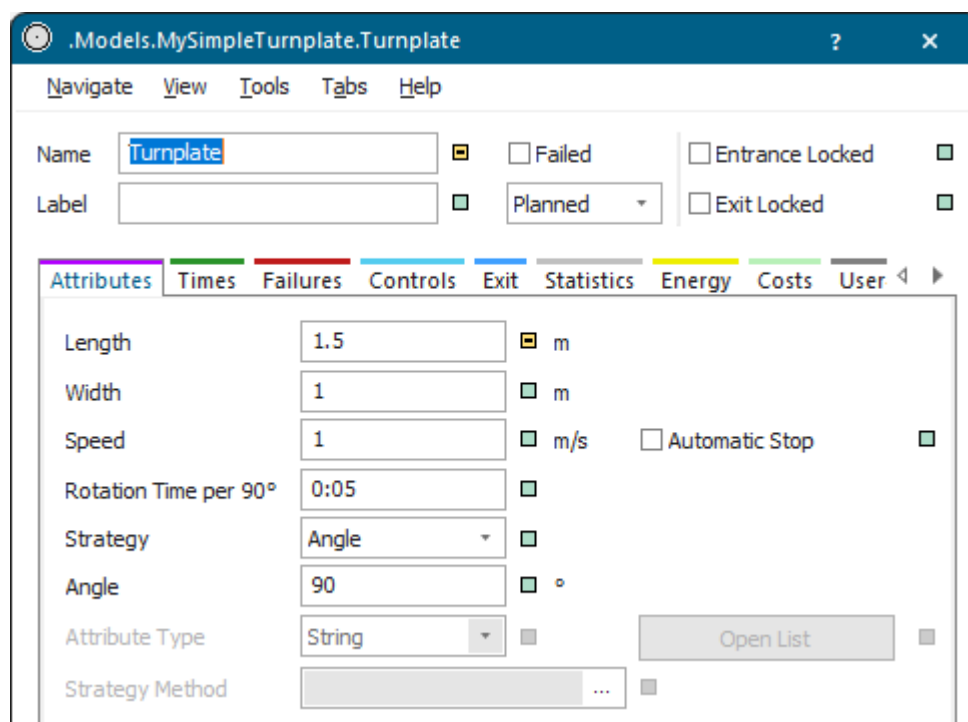


To create our simulation model, proceed as follows:

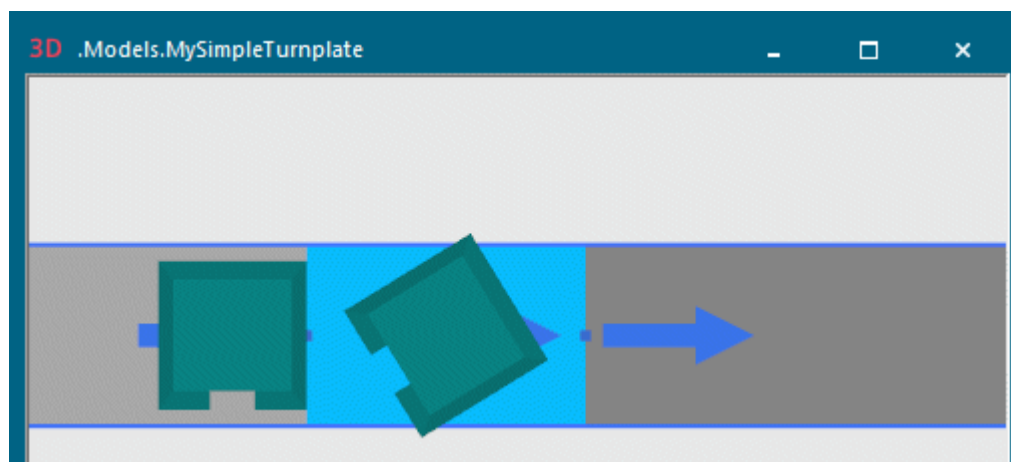
- First, configure the *Source*. As we want it to produce four parts only, we selected **Time of Creation > Number Adjustable** and entered 4 as the **Amount**. We then selected the part to be produced, namely *MyPartColored*, which we created in the folder *UserObjects*.



- Insert the *Conveyor* that feeds the parts to the *Turnplate*. You can make it as long as you like, you do not have to change any of the other settings though.
- Insert the *Turnplate* and the *Conveyor* that moves the parts on to the *Drain*.
- Configure the *Turnplate*. As we want it to rotate the part by an angle of 90 degrees, you can use the default setting, namely the **Strategy > Angle** and a rotation **Angle** of **90** degrees.



- Start the simulation and watch how the *Turnplate* rotates the part. You will not be able to see anything with the default setting for the **simulation Speed**, as the simulation runs too fast.
- To fix this, adjust the **Simulation Time** in the *EventController*.
- Start the simulation again and watch how the *Turnplate* rotates the part. You might want to further reduce the speed by dragging the slider in the *EventController* to the left.



Back to [Align and Shrink-Wrap Parts with the Turnplate](#)

Model a Shrink Wrapper

Model a Shrink Wrapper

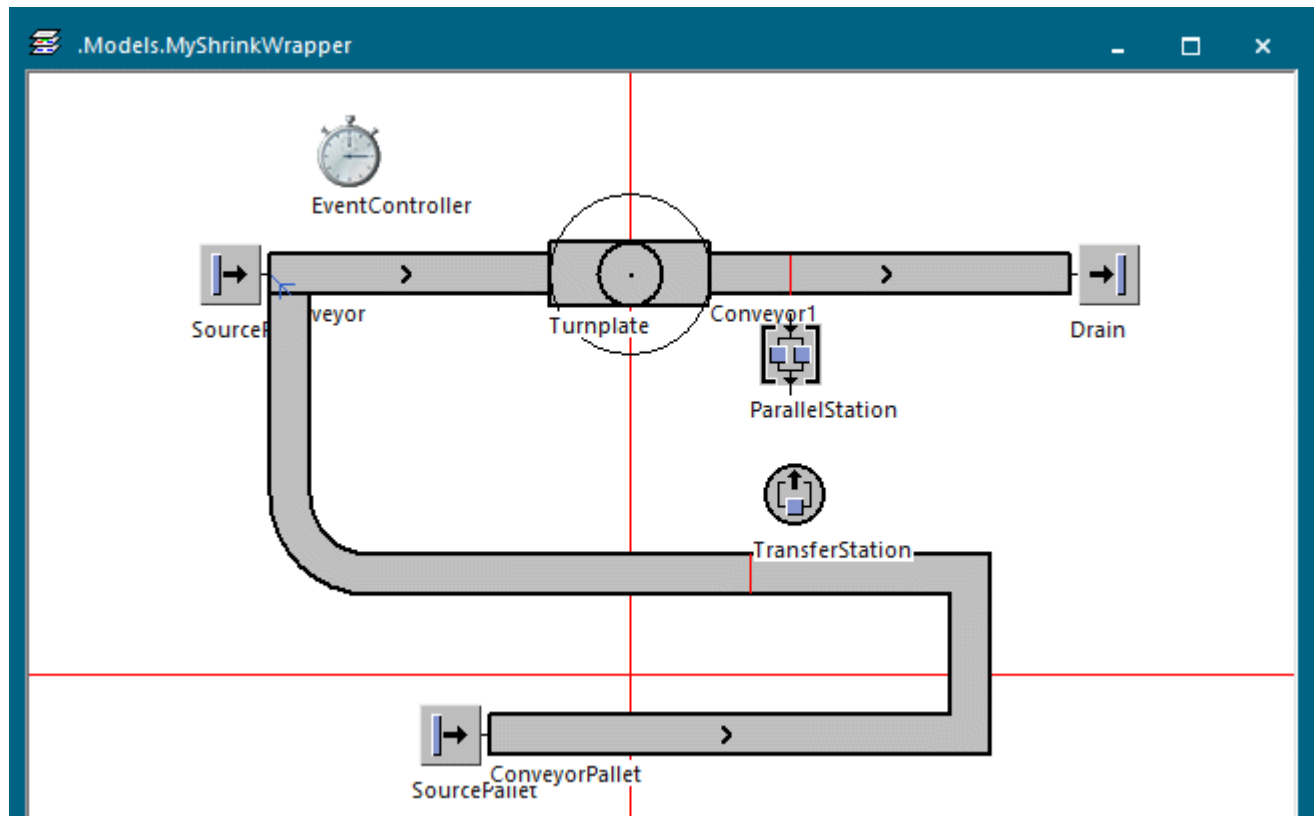
You can use the *Turnplate* to model a shrink wrapper. In our example we apply an overwrap around the parts on our pallet to stabilize the load.

In this sample model the *SourceParts* creates nine parts. It moves them onto *Conveyor1*, which feeds them to the *Turnplate* . The *Turnplate* turns them by 90 degrees to the right. The successor line moves the parts onto the *ParallelStation* , which processes them for two minutes. It then moves them on to the *TransferStation*, which loads them onto a pallet. The *ConveyorPallet* moves the pallet back to the *Turnplate*, which shrink wraps it, and moves it on to shipping, i.e., to the *Drain*.

To create our simulation model, proceed as follows:

- [Configure the Source and the Feeder Line](#)
- [Configure the Turnplate](#)
- [Configure the Conveyor Which Transfers the Parts](#)
- [Configure the Stations Which Handle the Pallet](#)
- [Adjust Shrinkwrapper and Parts](#)

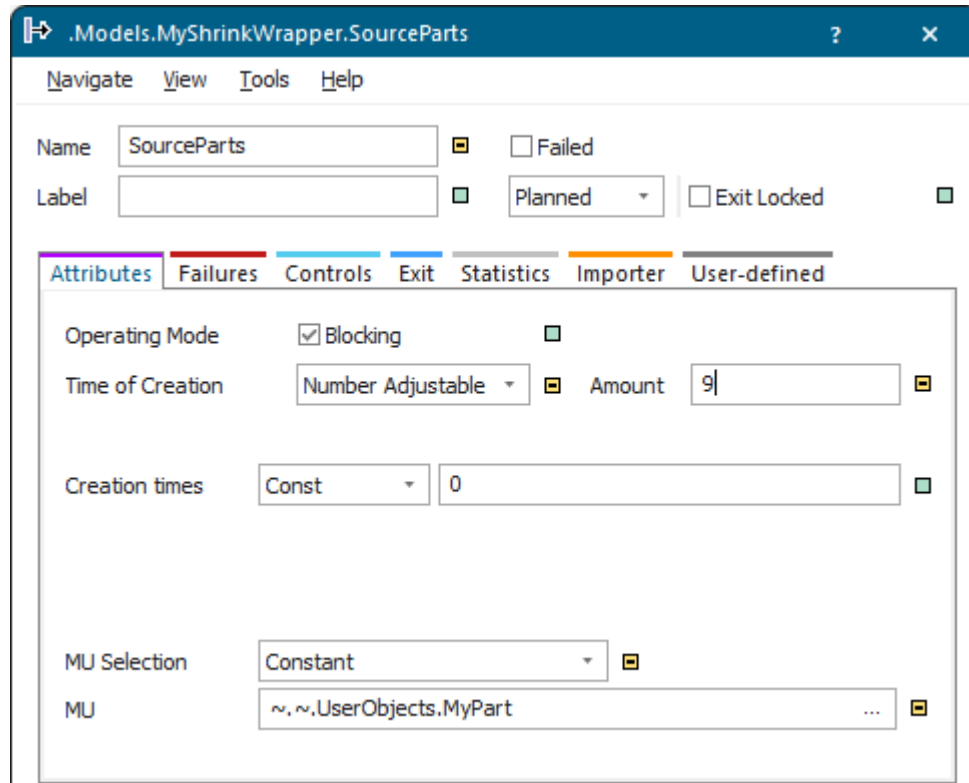
The finished simulation model looks like this in 2D:



Back to [Align and Shrink-Wrap Parts with the Turnplate](#)

Configure the Source and the Feeder Line

Insert and configure the *Source*. As we want it to produce nine parts only, we selected **Time of Creation > Number Adjustable** and entered 9 as the **Amount**. We then selected the part *MyPart*, which we created in the folder *UserObjects*.



Next, insert the *feeder line* that moves the part on to the *Turnplate*. You can use the *Conveyor* with its default settings.

Back to [Model a Shrink Wrapper](#)

Configure the Turnplate

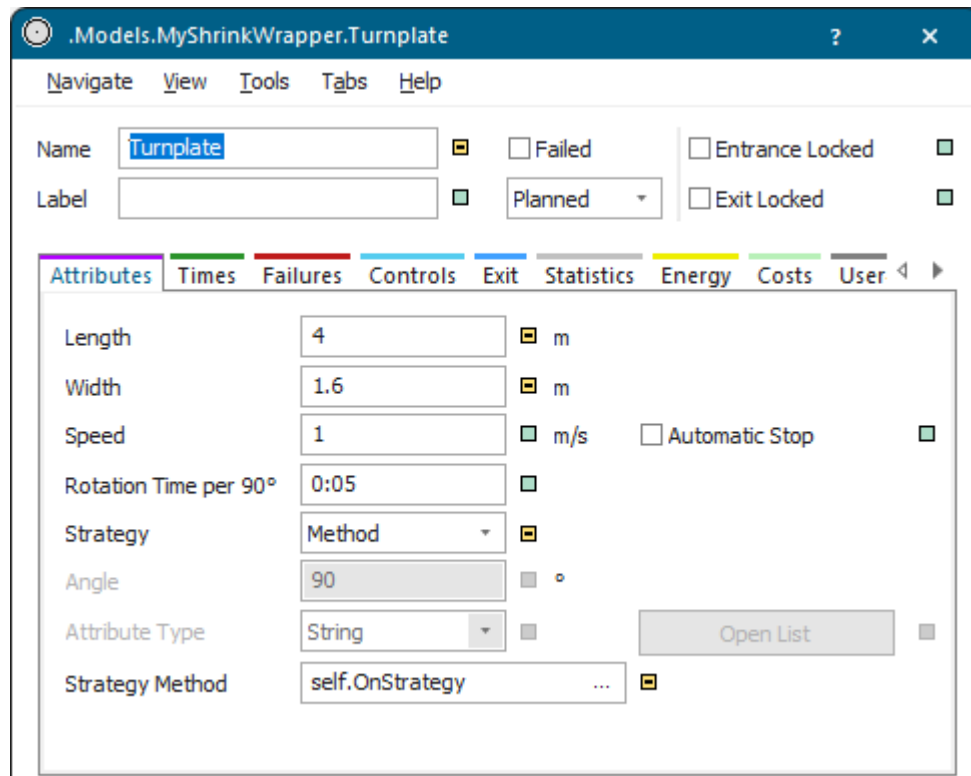
Insert the *Turnplate* and configure it. The *Turnplate* has two tasks in our example.

- It rotates the parts by 90 degrees to the right and then moves them on to *Conveyor1*.
- It rotates the pallet after the parts have been stacked onto it to simulate the shrink wrapper.

To accomplish this, we have to use a method:

- Select the **Strategy > Method**.
- Specify the name of the **Strategy Method** and program the actions it has to execute.

We decided to use a *user-defined attribute* of data type *method*. We right-clicked in the text box **Strategy Method** and selected **Create Control**. *Plant Simulation* then entered **self.OnStrategy** and opened the method.



We typed in the following source code, which first sets the rotation angle of the part itself and then of the pallet after the parts have been loaded onto it. The leading sign in the instruction - (4 * 360) designates the direction in which the *Turnplate* rotates; + stands for clockwise, - stands for counterclockwise. Our example rotates the pallet four times by 360 degrees counterclockwise.

```
var rotAngle: integer
if @.typeOf(.UserObjects.MyPart)           // rotates the part
    rotAngle := 90
else
    if @.typeOf(.UserObjects.MyPallet)      // rotates the pallet
        rotAngle := -(4 * 360)             // minus (-) designates
        counterclockwise rotation
    end
end
?.rotatePart(rotAngle)
```

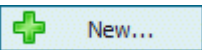
Back to [Model a Shrink Wrapper](#)

Configure the Conveyor Which Transfers the Parts

Insert and configure the *Conveyor* which receives the parts from the *Turnplate*.

All you have to do here is to define the sensor which triggers moving the parts to the *ParallelStation*.

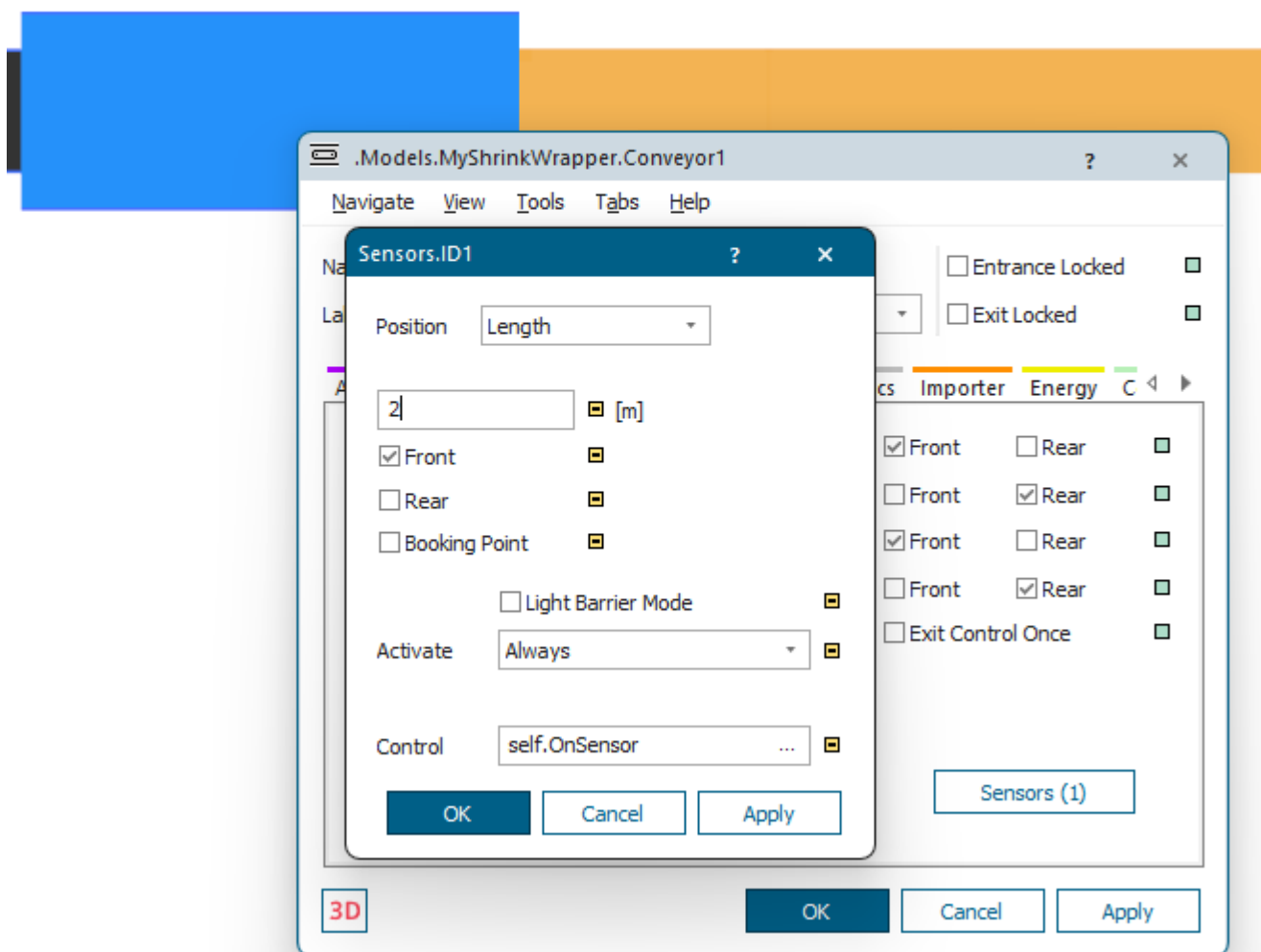
- Click the tab **Controls**.

- Click  and define a **length**-oriented sensor that is **front** triggered.
- Right-click into the text box **Control** and select **Create Control**.
- Copy the following source code and paste it. It determines how to move on the part or the pallet.

```
param sensorID: integer, front: boolean
if @.typeOf(.UserObjects.MyPart)
    @.move(ParallelStation)
end
```

Turnplate

Conveyor1

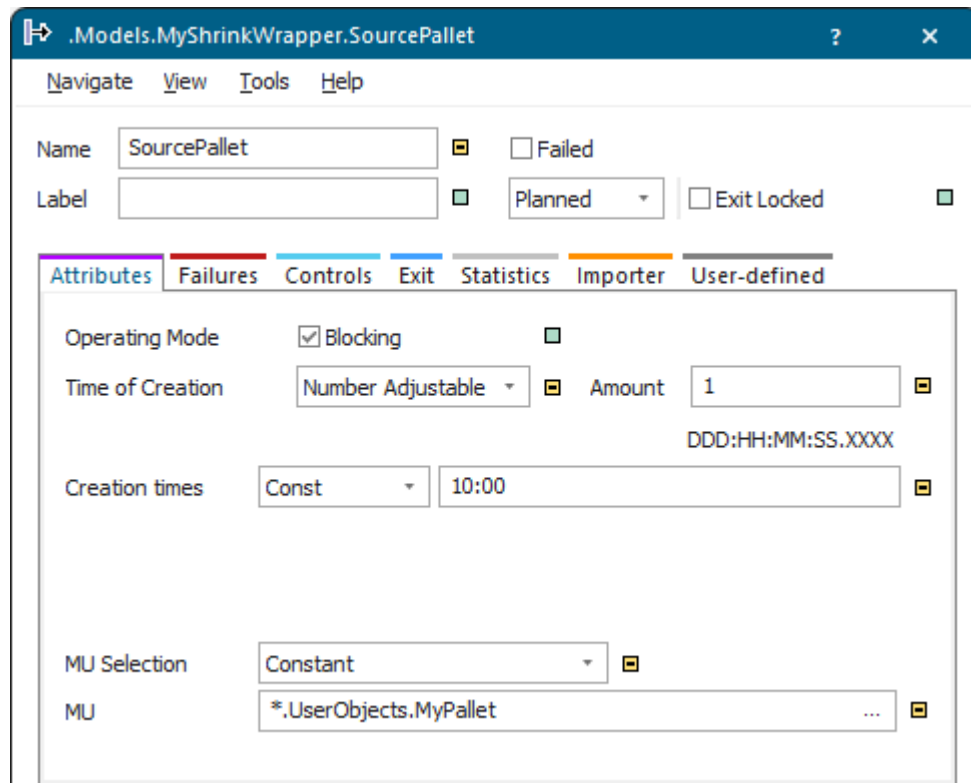


Back to [Model a Shrink Wrapper](#)

Configure the Stations Which Handle the Pallet

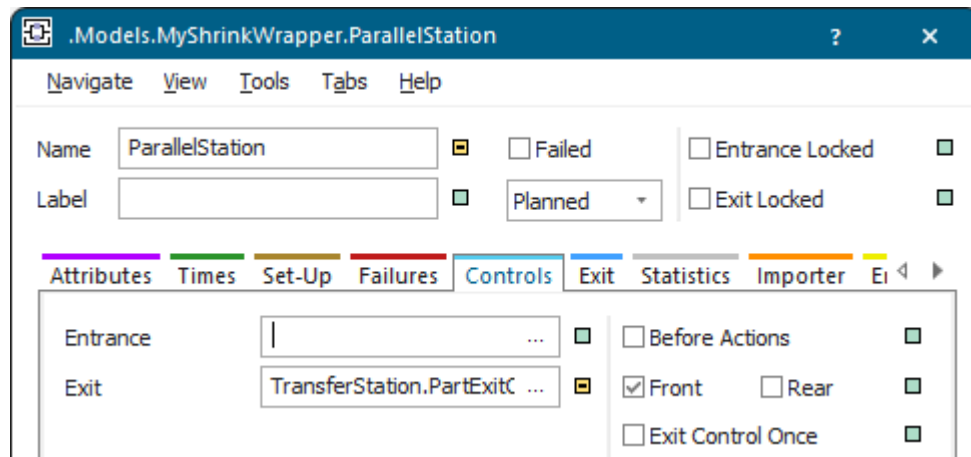
Finally, insert and configure the stations which create and handle the pallet onto which the parts will be loaded.

- First, insert and configure the *Source* which produces the pallet. To produce a single pallet after 10 minutes, select **Time of Creation > Number Adjustable** and enter an **Amount** of 1. Enter 10:00 as the **Creation Times**.

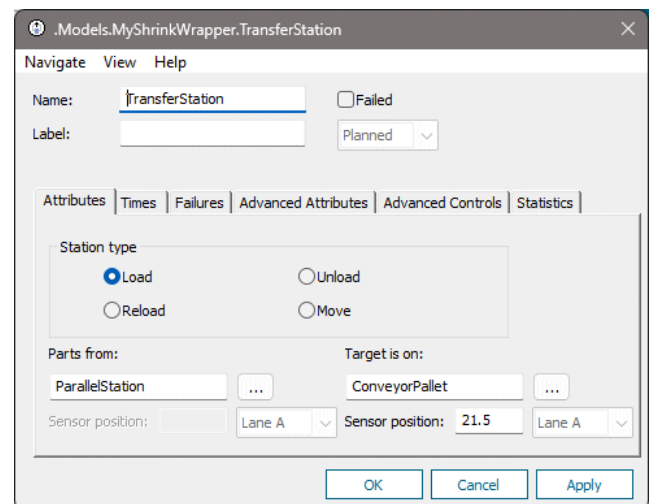
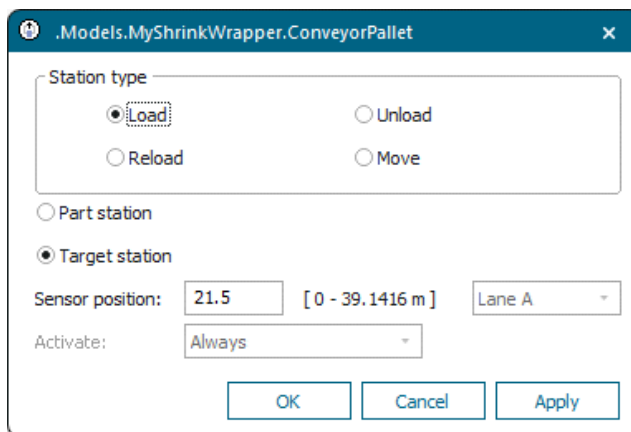


Then, select *MyPallet* as the **MU** to be produced.

- Next, insert the *ParallelStation* onto which *Conveyor1* moves the parts. You do not have to change any of its settings. The *TransferStation* will enter the **Exit Control** of the *ParallelStation* automatically once we configure the *TransferStation*.



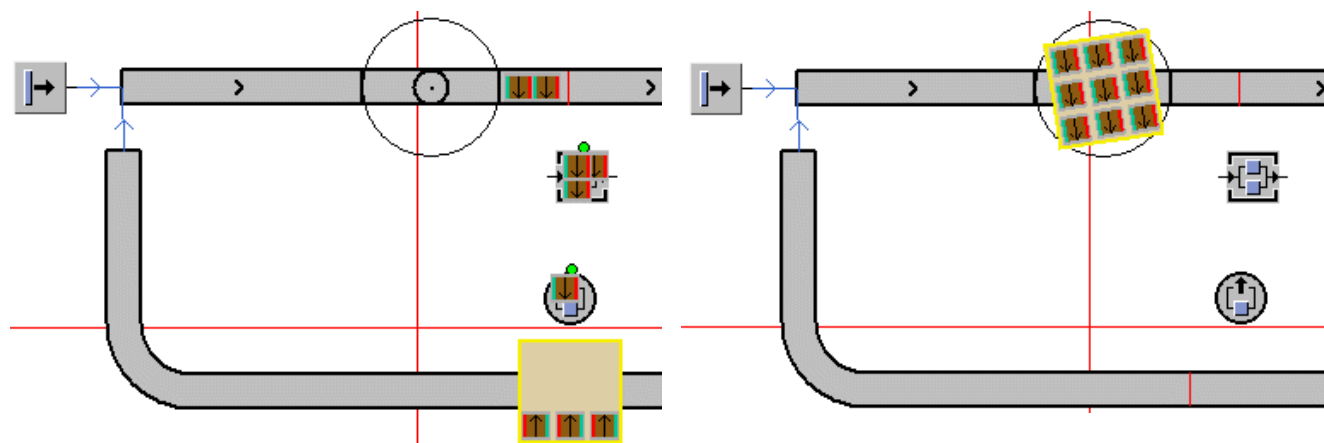
- Finally, insert and configure the *TransferStation*.
 - Drag the station from which the **parts** arrive, the *ParallelStation* in our case, onto the *TransferStation*. This enters the **Exit Control** into the *ParallelStation*.
 - Drag the station on which the **Target** of the loading action (the pallet), the *ConveyorPallet* in our case, is located onto the *TransferStation*. Select the station type **Load** and enter the position of the sensor on the *ConveyorPallet* and click **Apply**. Adjust the position until it is located at the position where it needs to be and click **OK**.



- Start the simulation and watch how the *Turnplate* rotates the part. You will not be able to see anything with the default setting for the **simulation speed**, as the simulation runs too fast.
- To fix this, you can reduce the simulation speed in a *reset method*.

```
EventController.Speed := 60
```

- Start the simulation again and watch how the parts are first rotated to the right, moved to the *ParallelStation*, and moved on to the *TransferStation*, which loads them onto the pallet. The loaded *pallet* is then fed back to the *Turnplate*, which rotates it four times around its center to the left and then moves it on to *Shipping*.

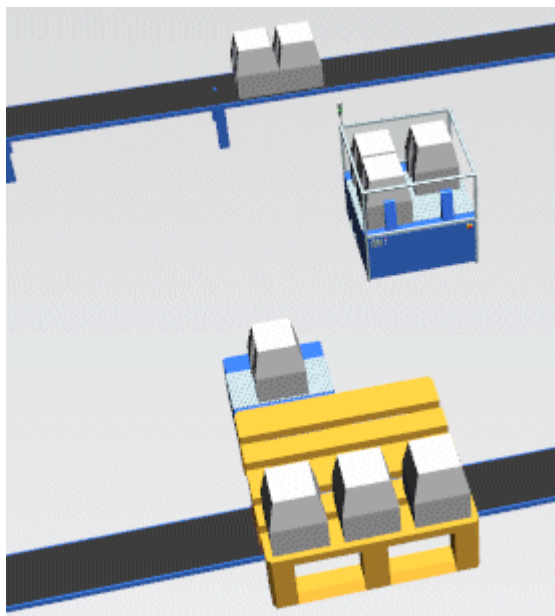


Back to [Model a Shrink Wrapper](#)

Adjust Shrinkwrapper and Parts

As we originally created the model in 2D in a previous version, we see that we have to adapt some settings, when we run the simulation model.

- The **parts** look strange.



As we created the model in a previous version, it uses the size of the part of that version:

The screenshot shows the configuration window for a part named 'MyPart'. The 'Attributes' tab is active, displaying the 'MU Size' section with the following values: Length 0.8 m, Width 0.8 m, and Height 0.1 m. The 'Booking Point' section shows three rows of values: 0.5 [0..1] (0.40 m), 0.5 [0..1] (0.40 m), and 0 [0..1] (0.00 m). The 'Conveying Direction' is set to 0 (forward).

MU Size	
Length	0.8 m
Width	0.8 m
Height	0.1 m

Booking Point	
0.5	[0..1] (0.40 m)
0.5	[0..1] (0.40 m)
0	[0..1] (0.00 m)

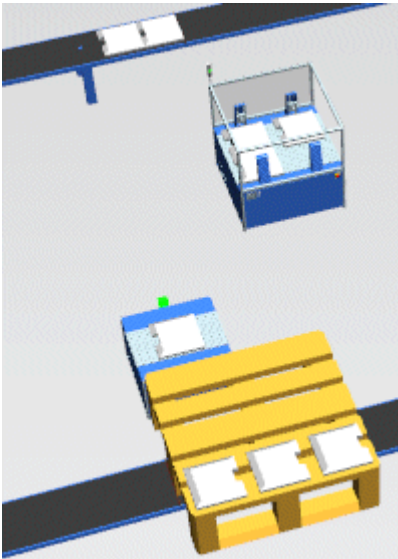
In the current version the part has these dimensions:

This screenshot is identical to the one above, showing the same configuration for 'MyPart' with Length 0.8 m, Width 0.8 m, and Height 0.1 m.

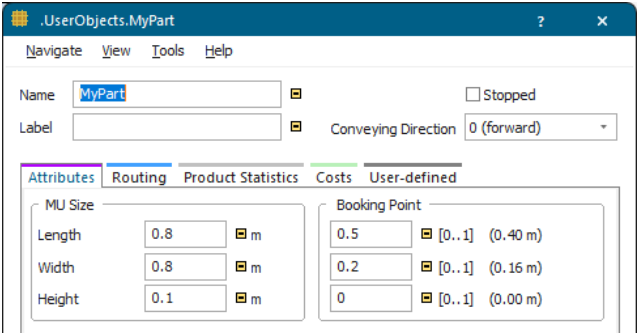
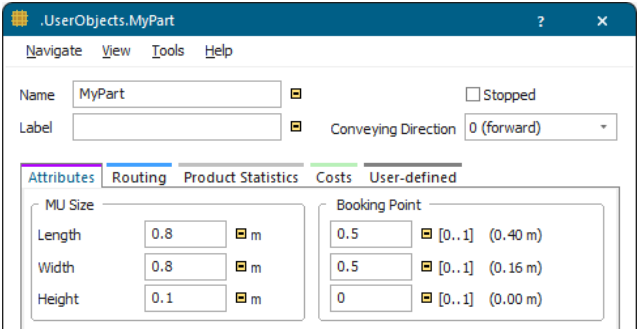
MU Size	
Length	0.8 m
Width	0.8 m
Height	0.1 m

Booking Point	
0.5	[0..1] (0.40 m)
0.5	[0..1] (0.40 m)
0	[0..1] (0.00 m)

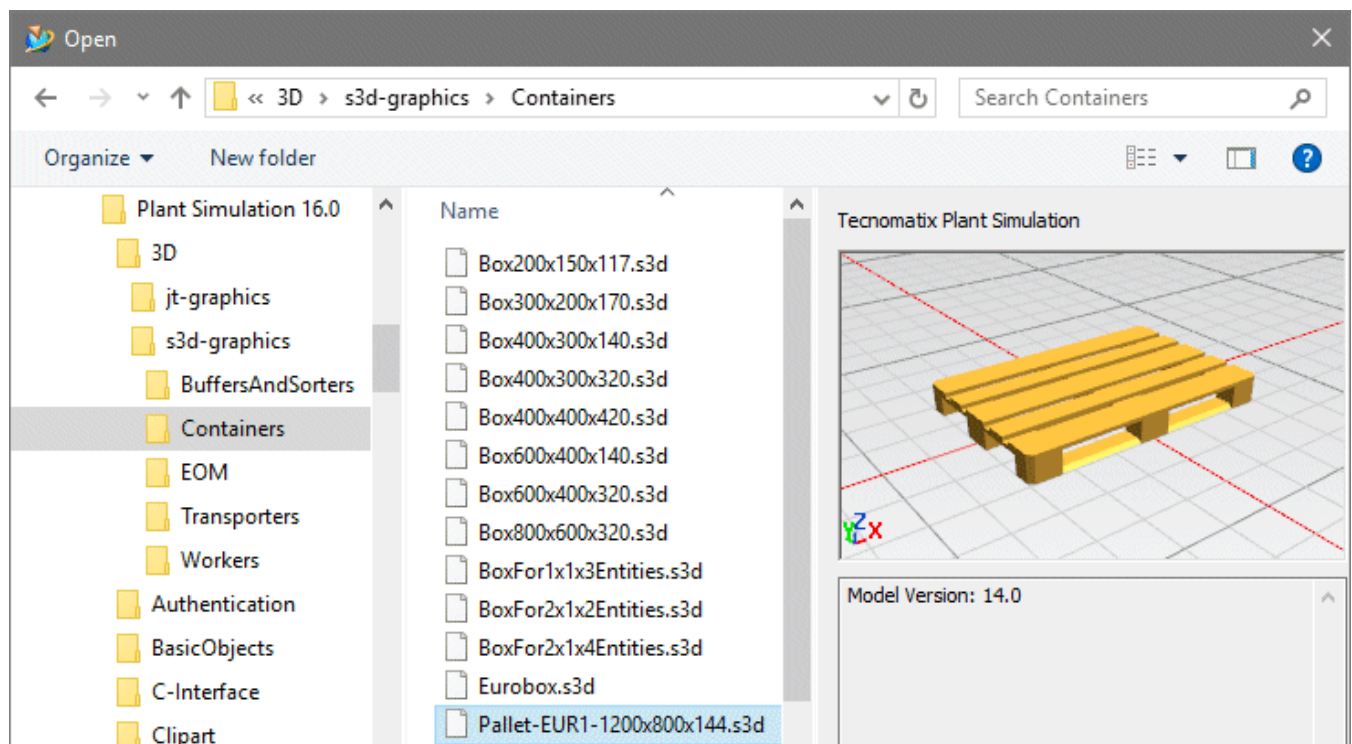
When we enter 0.1 meters as the height of our part, *MyPart*, and leave length and width at 0.8 meters, it looks much better:



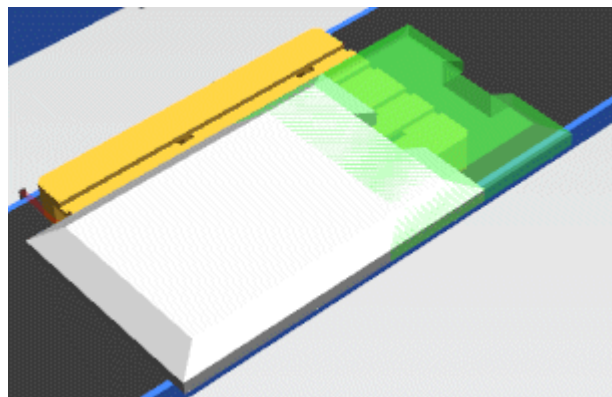
Note
If the parts are shown with an offset on the *ParallelStation* and on the *pallet*, check the position of the booking point of the part, compare **Booking Point Length** and **Booking Point Width**.

Booking point with an offset	Booking point centered
	
	

- The pallet is still too big in 3D though. To fix this, we replace its graphic in 3D, by opening *MyPallet* in 3D in the *Class Library* and selecting the context menu command **Exchange Graphics** in the 3D window. We use this EUR-pallet:

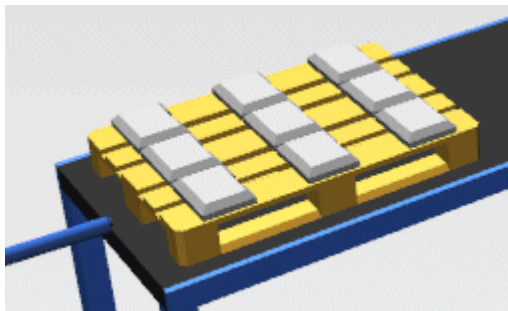


- The pallet now fits the width of the conveyor, the parts are much too big, they protrude the pallet and they overlap.

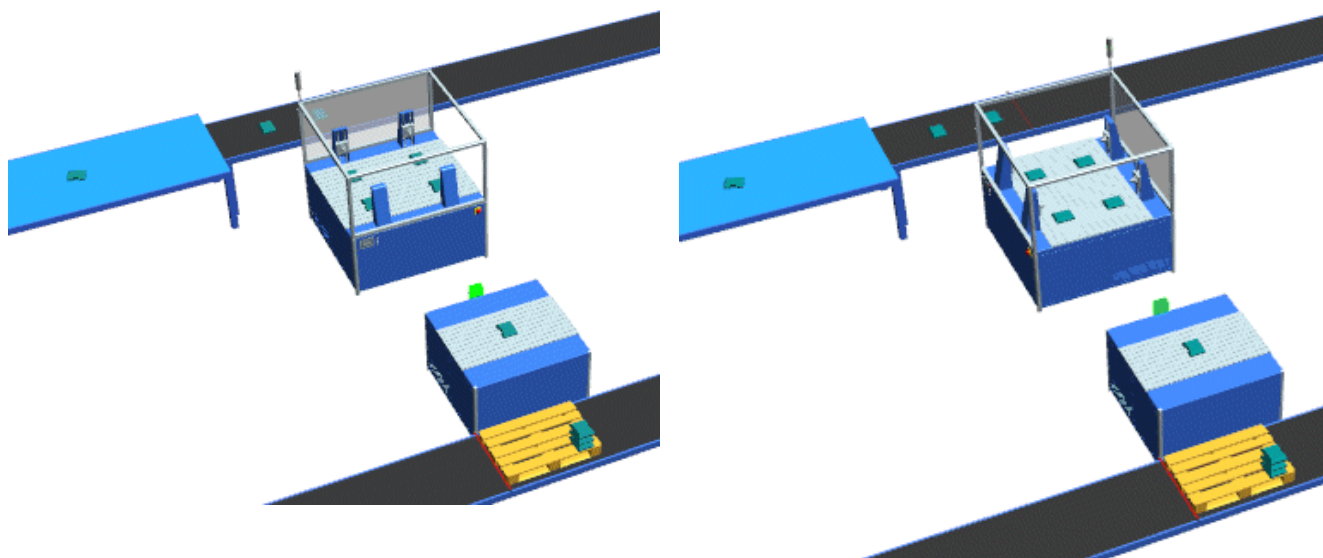


This is easy to fix though:

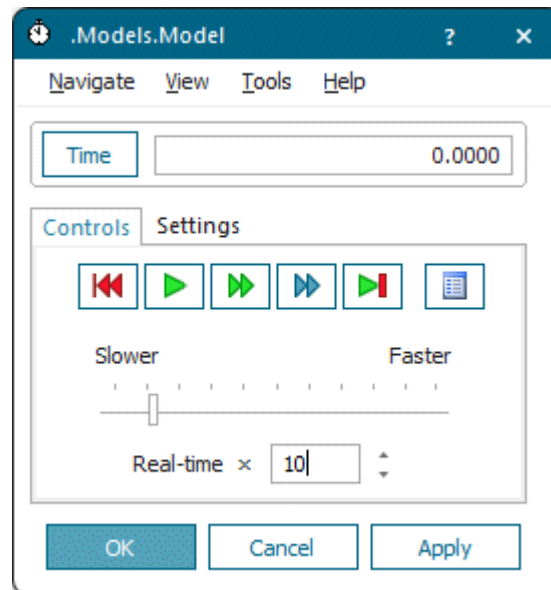
- Right-click *MyPart* in the *Class Library* and select **Edit 3D Properties**.
- Change to the tab **Transformation** and reduce the scaling. You might have to clear **Scale automatically** to show the settings. We selected a uniform scaling factor of **0.5**. This then looks like this:



- Then, we select a different color for the part in 3D to better see it in the model and adjust the positions of the objects in the model. To prevent the parts from breaking through the glass plates at the back and the front of the *ParallelStation*, we rotate it by 90 degrees.



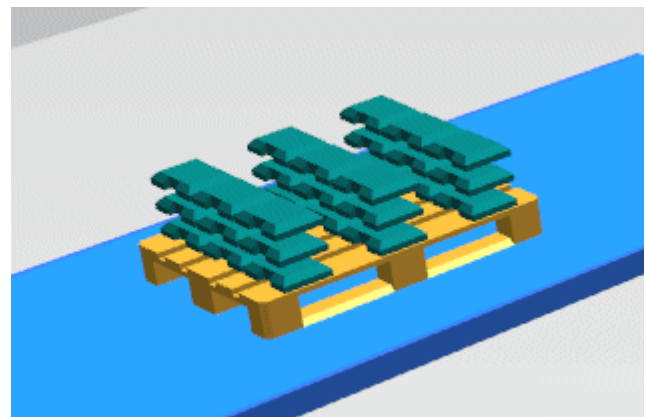
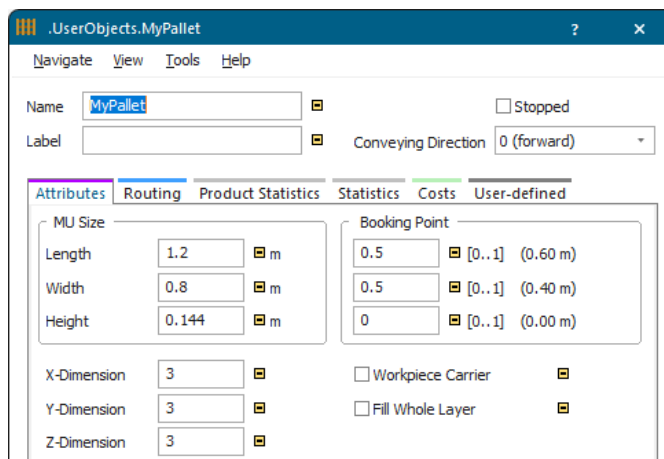
- We finally save the model as a **3D Only** model with a different name and run the simulation again. As we'd expect the part parts are produced, conveyed, and loaded onto the pallet. The last part is proceeding so fast though that we cannot follow the animation with our eyes. Proceed as follows to fix this:
 - Open the *EventController* and select a **Real-time** factor of 10.



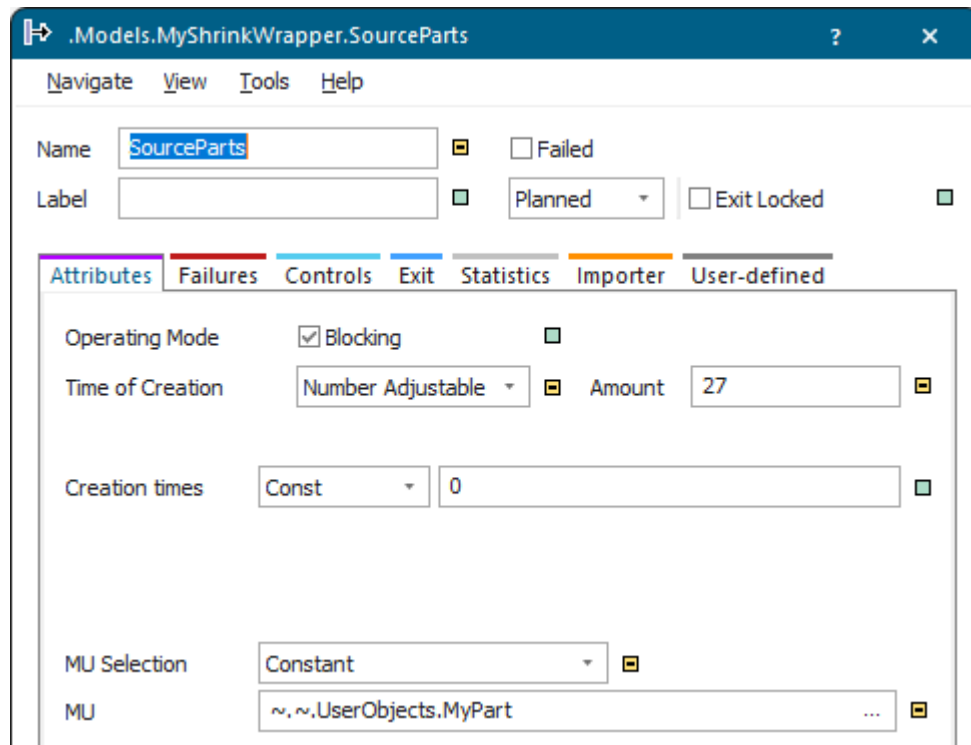
This also makes our *reset method* redundant and we can delete it.

- We then would have to wait too long for the simulation to continue with the original **Time of Creation** for the pallet of 10 minutes. For this reason we shorten the **Time of Creation** to 1 minute. We also adjusted the processing time of the *ParallelStation*. Now we can follow the simulation and the animation with our naked eye again.

We can also increase the capacity of the pallet in the **Z-Dimension** from 1 to 3 to stack the parts on it.



To facilitate this we also have to triple the number of parts that the *Source* named *SourceParts* produces.



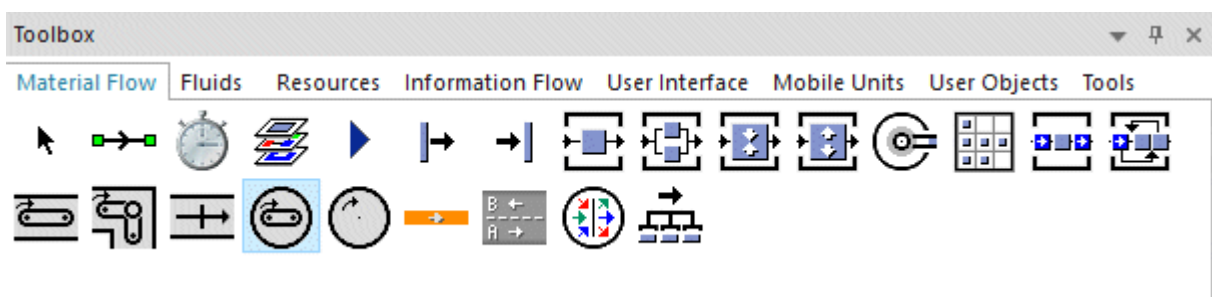
Back to [Model a Shrink Wrapper](#)

Move Parts On with the Turntable

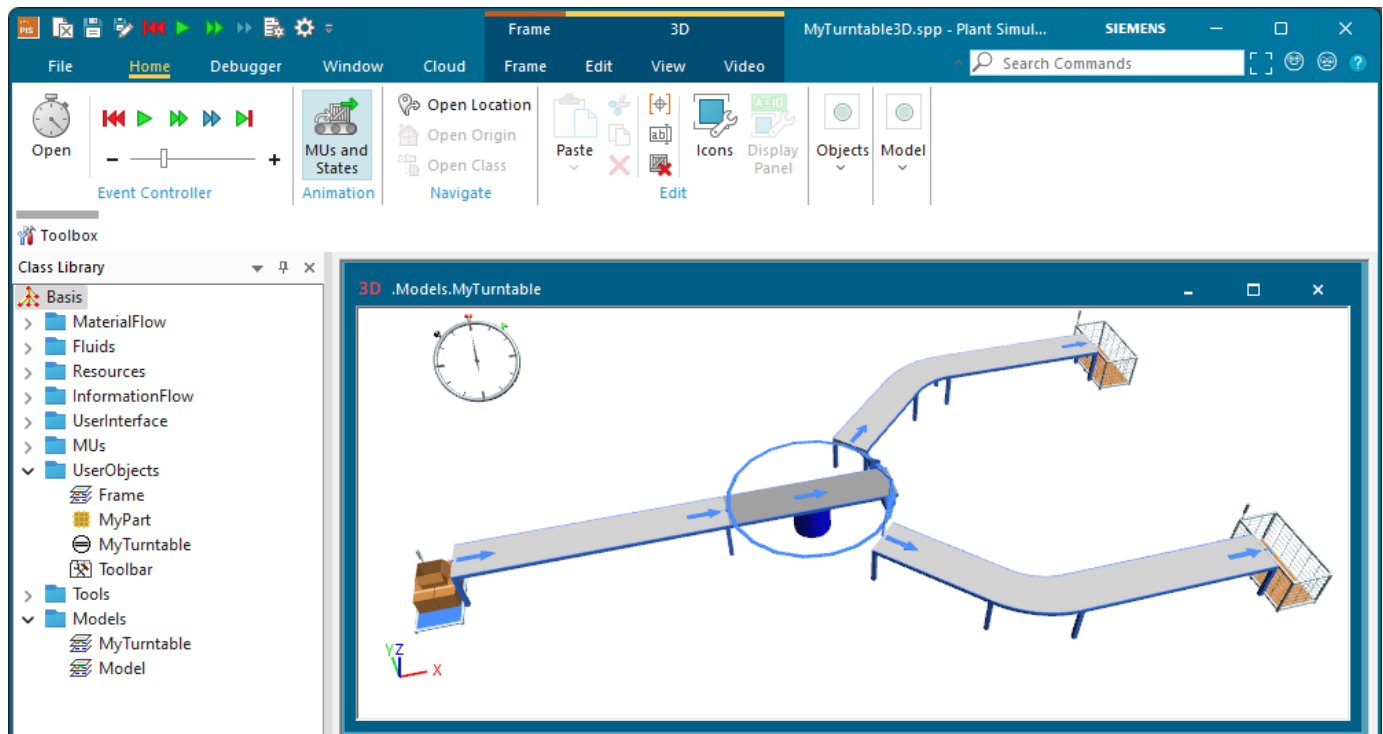
You can use the *Turntable* to model a rotating platform, which moves the part onto one of several connected material flow objects.

In the sample model the *Source* named *PartsIn* produces parts of type *MyPart* and moves a single part each on the feeder line of the *Turntable*. This in turn moves the parts **cyclically** on to its successors, i.e., to the feeder lines of the *Drains*, which we named *OutA* and *OutB* in our model.

You can insert the *Turntable* into your simulation model from the folder **MaterialFlow** in the *Class Library* or from the toolbar **Material Flow** in the *Toolbox*.

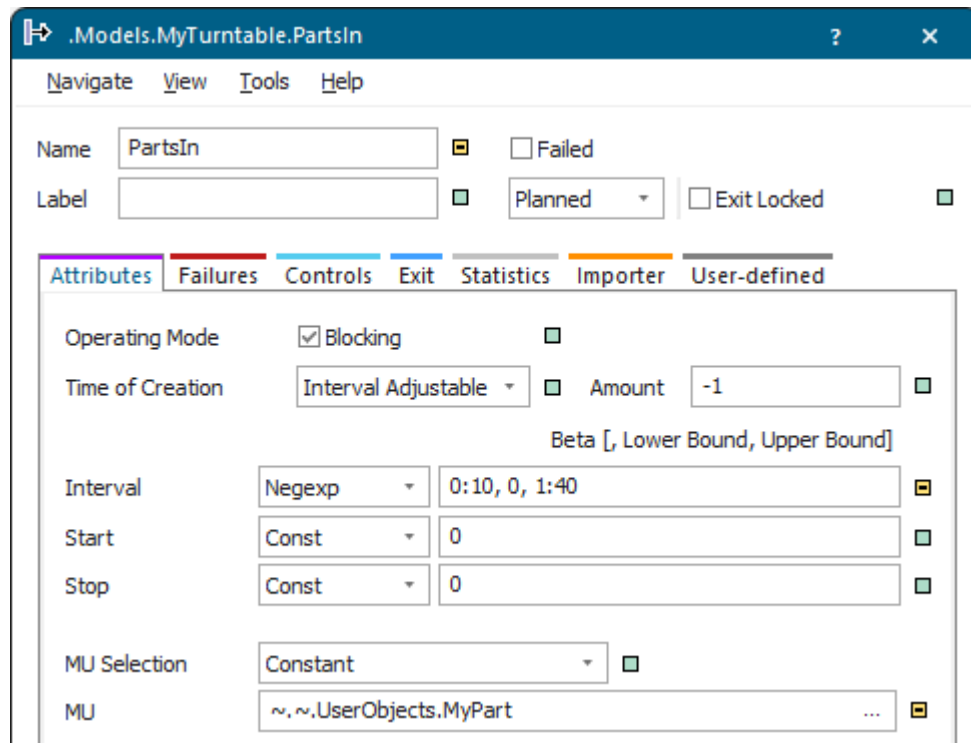


The finished simulation model looks like this:

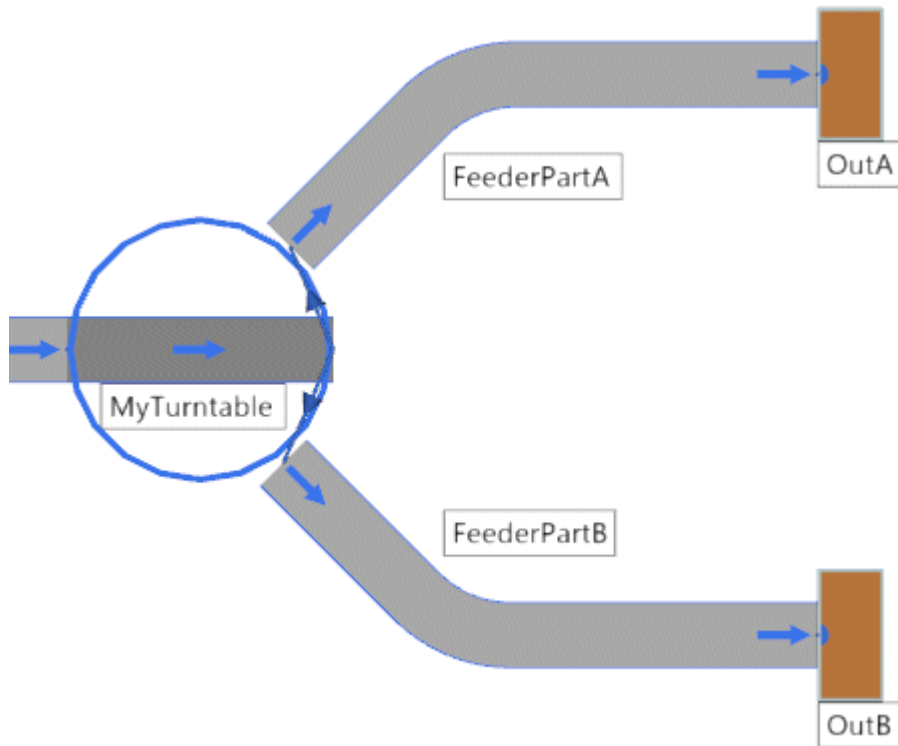


To create the simulation model, proceed as follows:

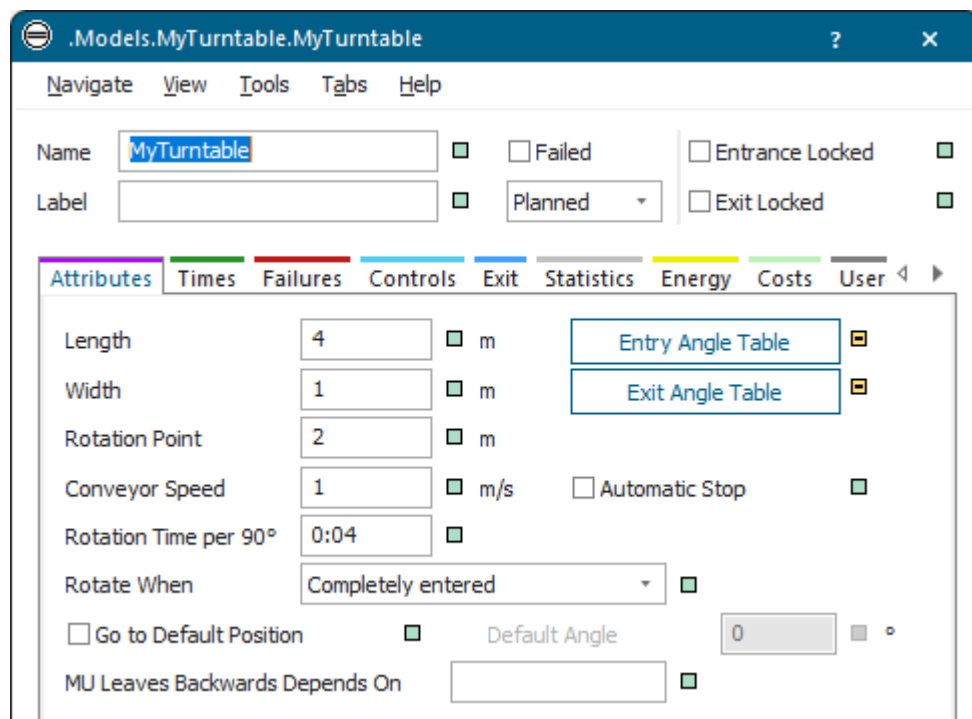
- First, configure the *Source*. Enter the interval during which it produces the parts and select the part it is to produce. We selected the **negative exponential** distribution, entered the parameters 0:10, 0, 1:40, and selected the part *MyPart*, which we created in the folder *UserObjects*.



- Insert the conveyor that feeds the parts to the *Turntable*. You can make it as long as you like, but you do not have to change any of the other settings.
- Insert the *Turntable* and its direct successors in the sequence of stations and connect them. The *Turntable* needs the successors to configure its **Entry Angle Table** and its **Exit Angle Table**. In our model these successors are the feeder lines of the *Drains*.



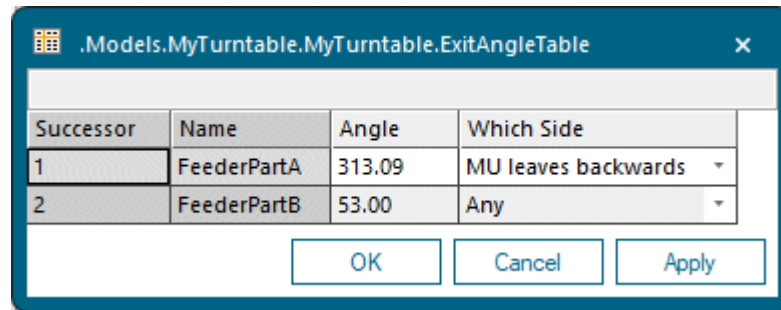
- Our *Turntable* is 4 meters long and has its **Rotation Point** in the center, i.e., at 2 meters.



- To connect the *Turntable* with its successors, select the *Connector* in the *Toolbox* and click the end point of the *Turntable*. Drag the mouse to the starting point of the *Conveyor* and click the left mouse button. You'll notice that the starting point of the *Connector* jumps to the left on the rotation circle of

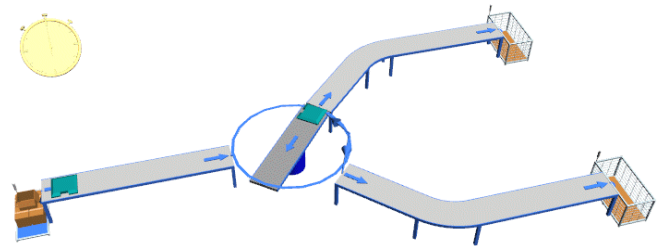
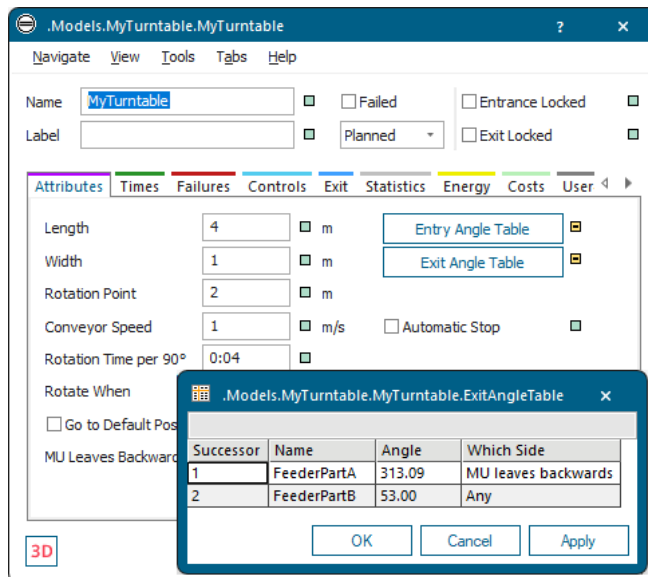
the *Turntable*. This is because *Plant Simulation* computes the angle at which the successor is located in the *Frame* in relation to the *Turntable*.

To view the exit angles, click **Exit Angles Table**.



Successor	Name	Angle	Which Side
1	FeederPartA	313.09	MU leaves backwards
2	FeederPartB	53.00	Any

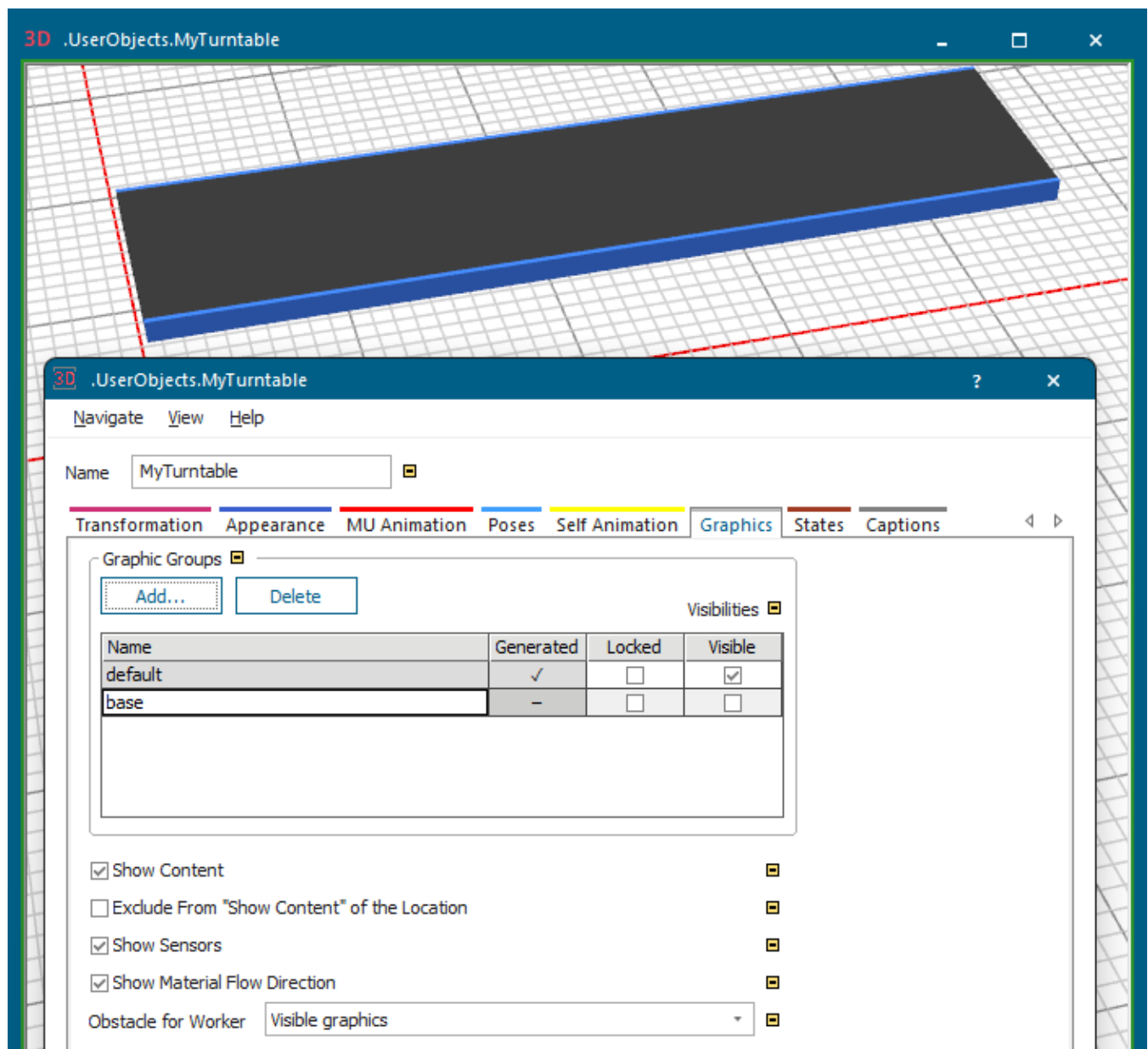
- Configure the *Drains* and start the simulation. Watch how the part moves onto the *Turntable*. It first rotates to the line *FeederPartA*, and moves the part onto it. It then returns to its original position, picks up the next part, rotates to the line *FeederPartA*, and moves the part onto it, and so on.
- To make the *Turntable* turn the part around before moving it onto the top feeder line, open the **Exit Angle Table** and select **Which Side > MU leaves backward** for the line *FeederPartA*. Start the simulation again and watch how the *Turntable* turns the part around.



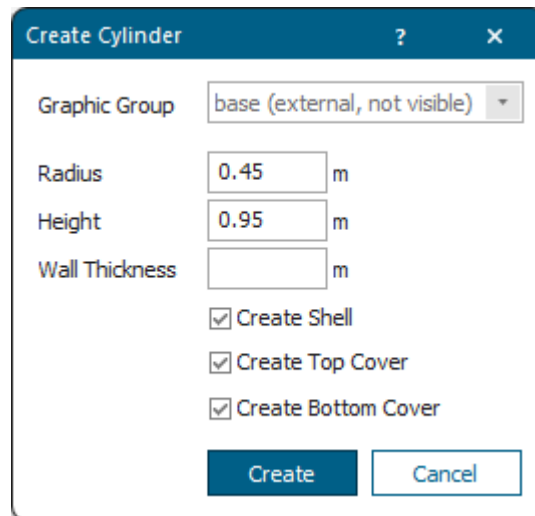
By default the *Turntable* is positioned in the air in 3D if you do not define any legs for it. If you do define legs, the entire *Turntable* rotates, including the legs. As this is not realistic, we create a cylindrical base on which the *Turntable* proper then rotates.

- Open the *Turntable* in a new window.
- Click the right mouse button and select **Edit 3D Properties**. Change to the tab **Graphics**. Click **Add** and create a new **graphic group** for the base of the *Turntable*. We named our graphic group **base**.

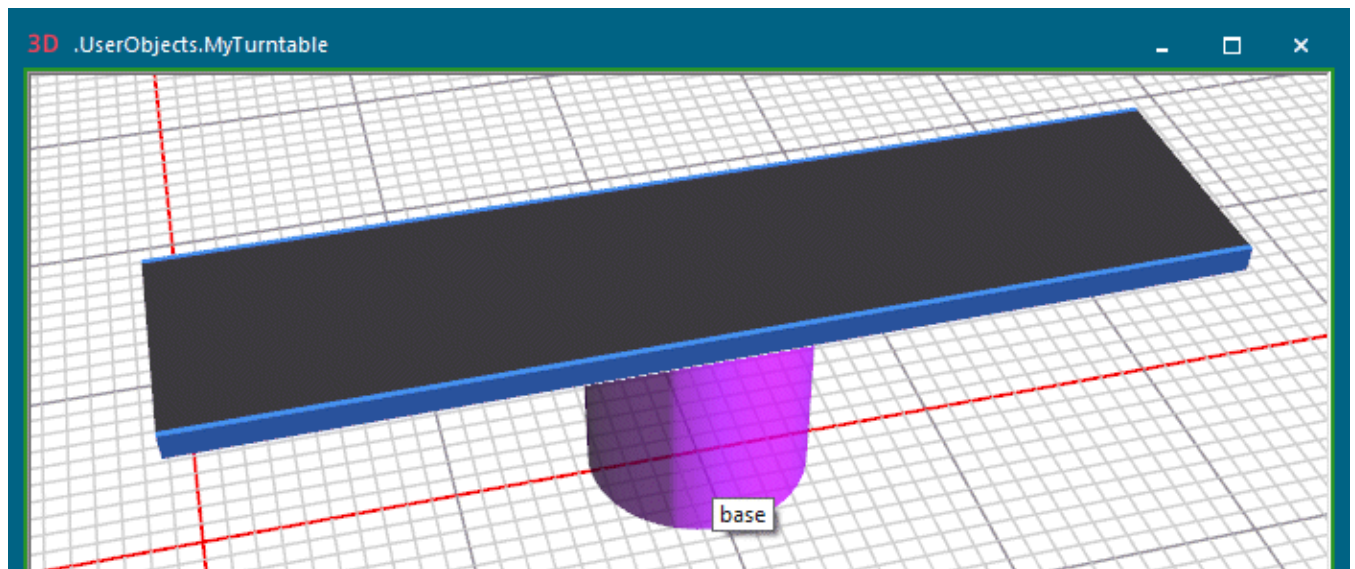
Click **OK**.



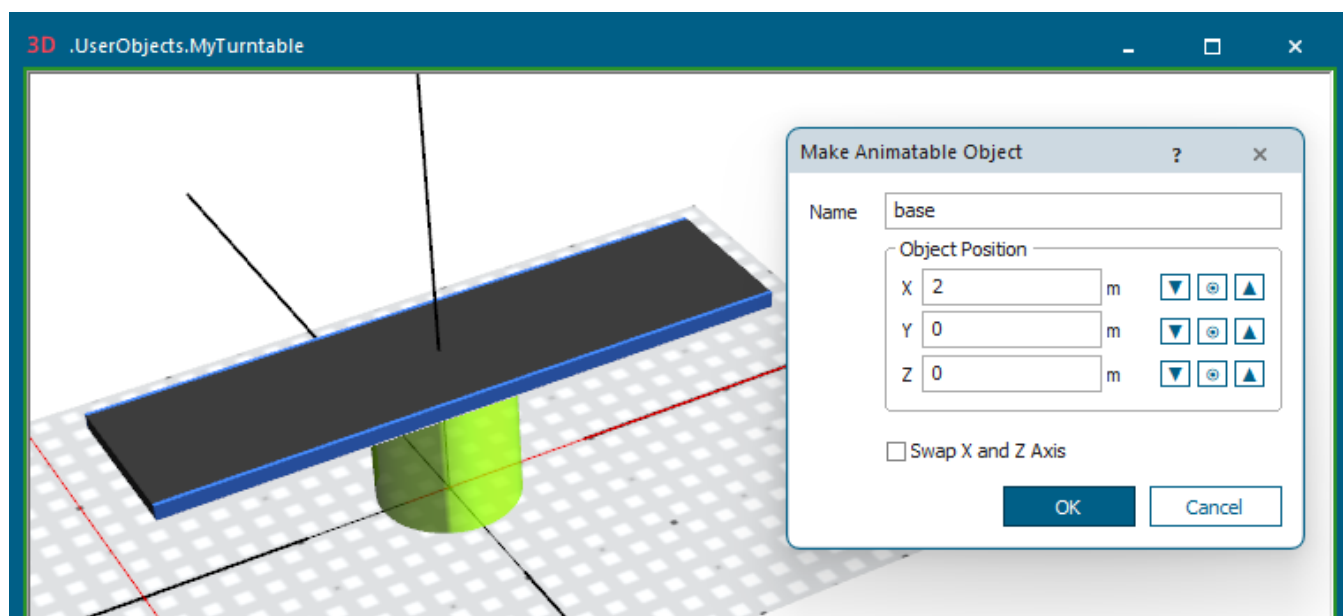
- Change to the ribbon tab **Edit**. Click **Cylinder** to create and insert a cylinder as the base of the *Turntable*. We selected these measurements.



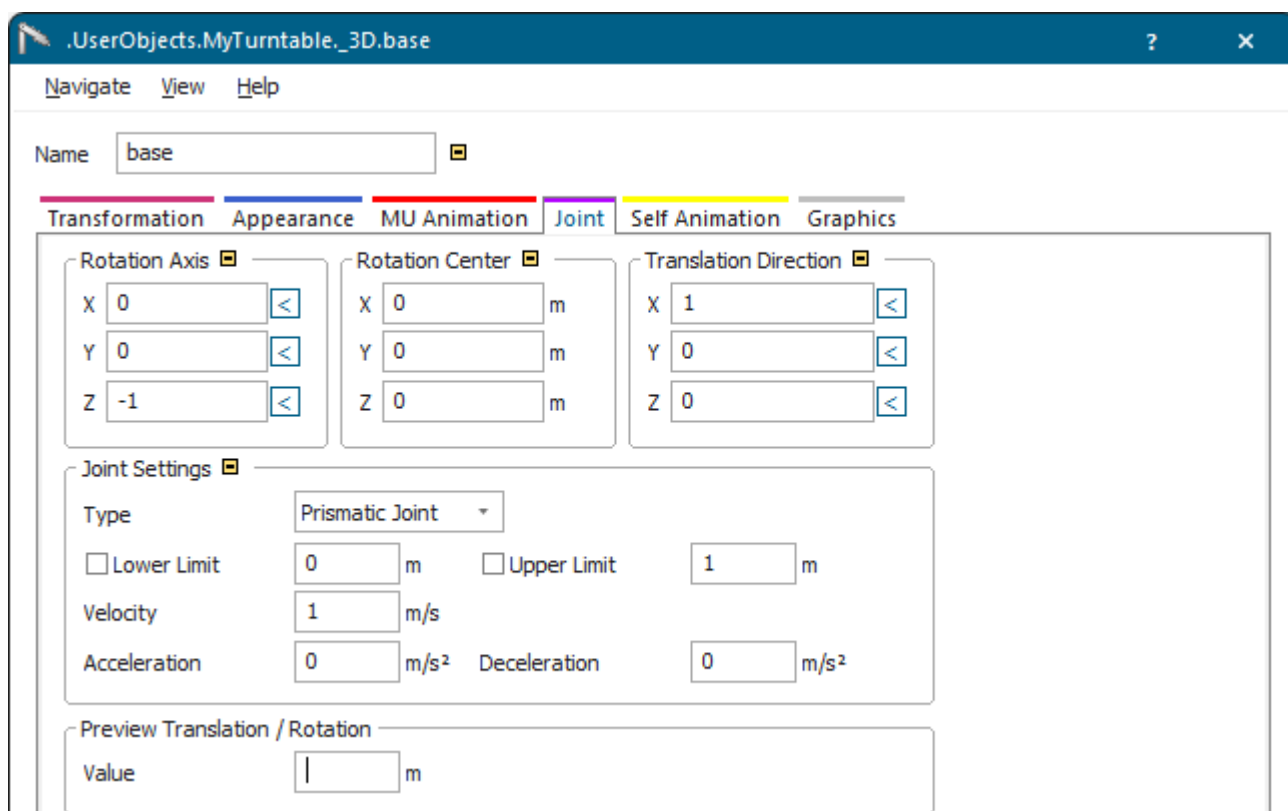
- Insert the base at the intersection of the grid line below the *Turntable* proper in its center.



- Click the base with the right mouse button and select **Make Animatable Object**. Type in base as the **name**. Click **OK** in the dialog.



- *Plant Simulation* opens the dialog **Edit 3D Properties** and selects the tab **Joint**. Select **Revolute Joint** and type in 0 as the **Velocity**. This prevents that the base below the *Turntable* also rotates when the *Turntable* proper rotates toward the feeder lines. Click **OK**.



- Run the simulation and watch what happens.